

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ  
ВЫСШЕГО ОБРАЗОВАНИЯ  
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

---

Кафедра защиты информации



Новосибирский  
государственный  
технический университет  
**НЭТИ**

**РАСЧЕТНО-ГРАФИЧЕСКАЯ РАБОТА**

**по дисциплине: «Программирование»**

Выполнил:

Студент гр. «АБ-522», «АВТФ»

*Никулин Р. Д.*

«\_\_\_» \_\_\_\_\_ 2026 г.

\_\_\_\_\_  
(подпись)

Проверил:

*доцент кафедры ЗИ*

*Архипова А. Б.*

«\_\_\_» \_\_\_\_\_ 2026 г.

\_\_\_\_\_  
(подпись)

Новосибирск 2026

## СОДЕРЖАНИЕ

|                                                             |    |
|-------------------------------------------------------------|----|
| Введение .....                                              | 3  |
| 1 Теоретические аспекты реализации методов шифрования ..... | 4  |
| 1.1 Алгоритм работы шифра простой замены .....              | 4  |
| 1.2 Алгоритм работы шифра Бофора .....                      | 6  |
| 1.3 Алгоритм работы шифра Тритемия .....                    | 8  |
| 2 Практическая часть .....                                  | 10 |
| 2.1 Техническое задание .....                               | 10 |
| 2.2 Характеристика задачи .....                             | 13 |
| 2.3 Алгоритм решения .....                                  | 14 |
| 2.4 Руководство пользователя .....                          | 17 |
| 2.5 Руководство системного программиста .....               | 23 |
| 2.6 Контрольный пример .....                                | 31 |
| Заключение .....                                            | 37 |
| Список использованных источников .....                      | 40 |
| Приложение А .....                                          | 42 |
| Приложение Б .....                                          | 43 |
| Приложение В .....                                          | 44 |

## ВВЕДЕНИЕ

Актуальность темы обусловлена тем, что криптографическое преобразование данных выполняется сегодня в каждой операционной системе и в каждом сетевом протоколе, а разбор классических шифров остается кратчайшим путем к пониманию того, как устроены современные. Отдельную ценность имеет прием, при котором реализация алгоритма отделена от программы, которая его вызывает: он позволяет заменять и добавлять алгоритмы, не изменяя и не пересобирая основной модуль.

Целью расчетно-графической работы является разработка консольной программы «Multi-Algo Cryptotool», которая шифрует и расшифровывает данные произвольного типа тремя классическими алгоритмами, размещенными в отдельных динамически загружаемых библиотеках, и создает ключи для этих алгоритмов.

Для реализации поставленной цели необходимо решить ряд задач: изучить три классических шифра и записать формулами их побайтовые варианты; спроектировать двоичный интерфейс между исполняемым модулем и библиотекой алгоритма; реализовать исполняемый модуль с разбором аргументов командной строки и потоковой обработкой данных; реализовать три библиотеки шифров и создание ключей криптостойким генератором операционной системы; написать тестовую утилиту; подготовить сценарии установки и удаления; оформить комплект программной документации.

Ожидаемым результатом является собранная и проверенная программа, исполняемый файл которой не содержит ни одной реализации шифра и у которой добавление нового алгоритма сводится к сборке новой библиотеки и к одной строке в перечне алгоритмов, а также комплект документов Единой системы программной документации.

Теоретической основой работы послужили учебные пособия и справочники по криптографии [6–8], из которых взяты определения и свойства шифров замены и многоалфавитных шифров, а также работы по истории

криптографии [9–11], из которых взяты сведения о происхождении простой замены, шифра Бофора и шифра Тритемия.

Практической основой выступили задание на расчетно-графическую работу и стандарты Единой системы программной документации [1–4], стандарт языка программирования [12] и описания системных интерфейсов операционной системы [14–16].

## **1 ТЕОРЕТИЧЕСКИЕ АСПЕКТЫ РЕАЛИЗАЦИИ МЕТОДОВ ШИФРОВАНИЯ**

В программе используются 3 метода шифрования: простая замена, шифр Бофора и шифр Тритемия. Все три относятся к классическим: первый является одноалфавитным шифром замены, второй и третий многоалфавитными [6].

Далее приняты единые обозначения. Открытые данные обозначены  $P$ , результат шифрования  $C$ , ключ  $K$ , абсолютный номер байта в потоке  $i$ , считая от нуля. Данные и ключ рассматриваются как последовательности байт, то есть целых чисел от 0 до 255, поэтому алфавитом каждого шифра служат 256 значений байта, а вся арифметика ведется по модулю 256. Такой выбор алфавита продиктован требованием технического задания обрабатывать файлы любых типов без искажения содержимого: при работе с буквами естественного языка байты, не попавшие в алфавит, пришлось бы либо пропускать, либо перекодировать. Все три шифра сохраняют длину данных: результат шифрования содержит ровно столько же байт, сколько содержали исходные данные.

### **1.1 Алгоритм работы шифра простой замены**

Простая замена, или одноалфавитный шифр замены, состоит в том, что каждый символ открытого текста заменяется другим символом того же алфавита по таблице, одинаковой для всего сообщения. Ключом служит сама таблица, то есть подстановка алфавита. Шифр известен с древности: его частными случаями являются древнееврейский атбаш, в котором алфавит

переворачивается, и шифр Цезаря, в котором алфавит сдвигается на три позиции [9, 10]. Произвольная подстановка алфавита, не сводящаяся к сдвигу или перевороту, применялась в дипломатической переписке Средних веков [11]. Первое известное описание способа вскрытия таких шифров частотным анализом дал в IX веке аль-Кинди [9].

В программе реализован побайтовый вариант шифра. Алфавитом служат 256 значений байта, поэтому ключом является таблица из 256 байт, в которой для каждого возможного значения байта записано значение замены. Правильным ключом считается только перестановка: каждое из 256 значений должно встречаться в таблице ровно один раз. Ключ проверяется перед каждым преобразованием подсчетом числа вхождений каждого значения; ключ, не являющийся перестановкой, отвергается. Шифрование сводится к выборке из таблицы:

$$C[i] = K[P[i]] \quad (1)$$

где  $P[i]$  — байт открытых данных с номером  $i$ ;

$C[i]$  — байт результата шифрования с тем же номером;

$K$  — ключ, таблица замены из 256 байт;

$K[b]$  — элемент таблицы с номером  $b$ , то есть значение, которым заменяется байт со значением  $b$ .

Расшифрование выполняется по обратной таблице. Обратной называется таблица  $K^{-1}$ , для которой  $K^{-1}[K[b]] = b$  при любом значении байта  $b$ :

$$P[i] = K^{-1}[C[i]] \quad (2)$$

где  $K^{-1}$  — таблица, обратная к таблице  $K$ ; строится один раз перед началом расшифрования: для каждого  $b$  в ее элемент с номером  $K[b]$  записывается  $b$ .

Расшифрование обращает шифрование потому, что ключ является перестановкой. Таблица задает отображение множества из 256 значений байта на себя, и если каждое значение встречается в таблице ровно один раз, отображение взаимно однозначно. Значит, у любого байта результата

существует единственный прообраз, обратная таблица определена однозначно, и подстановка формулы (1) в формулу (2) дает  $K^{-1}[K[P[i]]] = P[i]$ . Требование перестановки здесь не формальность: таблица, в которой хотя бы одно значение повторяется, необратима, поэтому библиотека такой ключ не принимает.

Стойкость шифра низка, хотя число ключей огромно: оно равно числу перестановок 256 элементов, то есть  $256!$ , что составляет примерно  $8,6 \cdot 10^{506}$ , или около  $2^{1684}$ . Перебор ключа при таком их числе невозможен, но он и не нужен. Замена одноалфавитная: одинаковые байты открытых данных всегда переходят в один и тот же байт результата, поэтому таблица частот байт результата является перестановкой таблицы частот байт исходных данных. Если известно, к какому типу относятся исходные данные, и известно распределение частот байт для этого типа, таблица восстанавливается сопоставлением частот, а оставшиеся элементы доопределяются по смыслу; это и есть частотный анализ [7, 9]. Для текстовых данных вскрытие тем надежнее, чем длиннее шифротекст. Кроме того, каждый известный байт открытых данных немедленно выдает один элемент таблицы: зная фрагмент исходного файла, например заголовок формата, противник получает соответствующие элементы ключа даром [6].

## 1.2 Алгоритм работы шифра Бофора

Шифр Бофора назван по имени Фрэнсиса Бофорта (1774–1857), ирландца по рождению, контр-адмирала британского флота и гидрографа; описание способа вместе с таблицей было издано отдельной карточкой в 1857 году [9]. Шифр является многоалфавитным и представляет собой вариант шифра Виженера с той разницей, что буква открытого текста не прибавляется к букве ключа, а вычитается из нее. Для алфавита из 26 букв правило шифрования записывается как  $C = (K - P) \bmod 26$  [8]. Ключом служит короткое слово, которое мысленно повторяется вдоль всего сообщения. Главная особенность шифра состоит в том, что он взаимен: расшифрование

выполняется той же самой операцией, что и шифрование, поэтому при ручной работе не нужны две разные таблицы и исключается часть ошибок [9].

В программе реализован побайтовый вариант. Алфавитом служат 256 значений байта, ключом последовательность из 16 байт, которая повторяется вдоль всех данных: байту с номером  $i$  соответствует байт ключа с номером  $i \bmod 16$ . Правильным ключом является любая последовательность из 16 байт, поэтому создание ключа сводится к чтению 16 байт у генератора операционной системы. Каждый байт данных вычитается из соответствующего ему байта ключа:

$$C[i] = (K[i \bmod 16] - P[i]) \bmod 256 \quad (3)$$

где  $P[i]$  — байт открытых данных с номером  $i$ ;

$C[i]$  — байт результата шифрования с тем же номером;

$K$  — ключ, последовательность из 16 байт;

$K[j]$  — байт ключа с номером  $j$ ;

$i \bmod 16$  — остаток от деления номера байта на длину ключа, то есть номер байта ключа, который приходится на этот байт данных;

$\bmod 256$  — приведение результата к диапазону от 0 до 255.

Расшифрование выполняется по формуле, отличающейся от формулы (3) только тем, что на месте открытых данных стоит результат шифрования:

$$P[i] = (K[i \bmod 16] - C[i]) \bmod 256 \quad (4)$$

Расшифрование обращает шифрование потому, что преобразование является инволюцией: применение его дважды возвращает исходное значение. Действительно, подстановка формулы (3) в формулу (4) дает  $(K[i \bmod 16] - (K[i \bmod 16] - P[i])) \bmod 256 = P[i]$ , поскольку оба слагаемых с ключом взаимно уничтожаются. По этой причине в библиотеке функция расшифрования не содержит собственного кода и вызывает функцию шифрования с теми же аргументами.

Стойкость шифра низка. Ключ короткий и повторяется, поэтому байты, номера которых дают одинаковый остаток от деления на 16, зашифрованы одним и тем же байтом ключа и образуют 16 групп, внутри каждой из

которых шифр вырождается в одноалфавитный. Длина ключа определяется по шифротексту методом Касиски или по индексу совпадений, после чего каждая из 16 групп вскрывается частотным анализом [7, 8]. Еще быстрее шифр вскрывается при известном открытом тексте: из формулы (3) следует  $K[i \bmod 16] = (C[i] + P[i]) \bmod 256$ , поэтому 16 подряд идущих известных байт открытых данных дают весь ключ целиком. Наконец, повторное применение одного и того же ключа к разным данным позволяет исключить ключ вычитанием шифротекстов.

### 1.3 Алгоритм работы шифра Тритемия

Шифр Тритемия назван по имени немецкого аббата Иоганна Тритемия (1462–1516), автора сочинения «Polygraphiae libri sex», напечатанного в 1518 году и считающегося первой изданной книгой по криптографии [9, 11]. В этом сочинении приведена квадратная таблица, в которой каждая следующая строка представляет собой алфавит, сдвинутый на одну позицию относительно предыдущей. Такая таблица получила название *tabula recta* и легла в основу всех позднейших многоалфавитных шифров, включая шифр Виженера и шифр Бофора [10]. Сам шифр Тритемия состоит в том, что первая буква текста шифруется по нулевой строке таблицы, вторая по первой, третья по второй и так далее, то есть сдвиг нарастает вдоль текста:  $C = (P + i) \bmod 26$ . Ключа в исходном шифре нет вовсе, его роль играет номер буквы.

В программе реализован побайтовый вариант, в котором нарастающий сдвиг дополнен ключом. Алфавитом служат 256 значений байта, ключом последовательность из 16 байт, повторяющаяся вдоль данных так же, как в шифре Бофора. К каждому байту данных прибавляется байт ключа и, дополнительно, номер байта в потоке:

$$C[i] = (P[i] + K[i \bmod 16] + i) \bmod 256 \quad (5)$$

где  $P[i]$  — байт открытых данных с номером  $i$ ;  
 $C[i]$  — байт результата шифрования с тем же номером;  
 $K$  — ключ, последовательность из 16 байт;  
 $K[j]$  — байт ключа с номером  $j$ ;  
 $i$  — абсолютный номер байта в потоке, считая от нуля; он же  
слагаемое, задающее нарастающий сдвиг;  
 $\text{mod } 256$  — приведение результата к диапазону от 0 до 255.

Расшифрование выполняется вычитанием тех же двух величин:

$$P[i] = (C[i] - K[i \bmod 16] - i) \bmod 256 \quad (6)$$

Расшифрование обращает шифрование потому, что при шифровании к байту прибавляется, а при расшифровании вычитается одна и та же величина  $K[i \bmod 16] + i$ . Эта величина однозначно определяется ключом и номером байта, а номер байта известен обеим сторонам: он равен смещению байта от начала потока. Подстановка формулы (5) в формулу (6) дает  $(P[i] + K[i \bmod 16] + i - K[i \bmod 16] - i) \bmod 256 = P[i]$ . Сложение и вычитание по модулю 256 выполняются над беззнаковыми восьмибитными значениями, поэтому перенос через границу байта определен и совпадает со взятием остатка.

Стойкость шифра низка и по существу не превышает стойкости шифра Бофора. Слагаемое  $i$  не является секретом: оно вычисляется из положения байта и известно всякому, кто видит шифротекст. Вычтя из каждого байта шифротекста его номер, противник получает шифротекст многоалфавитного шифра с периодом 16, который вскрывается методом Касиски и частотным анализом так же, как в 1.2 [7]. При известном открытом тексте ключ находится сразу:  $K[i \bmod 16] = (C[i] - P[i] - i) \bmod 256$ , и 16 подряд идущих известных байт дают весь ключ. Нарастающий сдвиг лишь маскирует повторы: он приводит к тому, что одинаковые байты исходных данных переходят в разные байты результата, из-за чего распределение частот в шифротексте выравнивается и одноалфавитность становится менее заметной, но секретности не добавляет.

## **2 ПРАКТИЧЕСКАЯ ЧАСТЬ**

### **2.1 Техническое задание**

Задание на расчетно-графическую работу требует разработать консольное приложение, которое шифрует и расшифровывает данные произвольного типа алгоритмом, выбранным пользователем при запуске, и создает ключи для этих алгоритмов. Ниже перечислены требования задания, существенные для дальнейшего изложения; они приведены в сжатом виде и своими словами. Полный текст требований содержится в отдельном документе «Техническое задание», оформленном по ГОСТ 19.201-78 [1] и входящем в комплект документации на изделие.

**Архитектура.** Исполняемый модуль отделяется от реализаций алгоритмов, каждый алгоритм размещается в собственной динамически подключаемой библиотеке. Взаимодействие строится по правилам двоичного интерфейса языка С, для передачи данных применяются структуры-обертки над указателем и размером буфера. Память для обмена выделяет исполняемый модуль. Исполняемому модулю запрещено делать предположения о свойствах загруженного алгоритма: длину ключа и имя алгоритма он обязан запрашивать у библиотеки. Перечень алгоритмов и соответствующие им базовые имена библиотек задаются в исходном коде исполняемого модуля, а имя файла библиотеки формируется во время выполнения в зависимости от операционной системы.

**Данные.** Обрабатываются файлы любых типов без искажения содержимого. Внутри программы данные и ключи хранятся и передаются только как массивы байт; строковые типы для этого не применяются. Файлы и стандартные потоки открываются в двоичном режиме. Ключи создаются, читаются, сохраняются и передаются в двоичном виде.

**Режимы работы и интерфейс.** Программа работает в трех взаимоисключающих режимах: информационно-справочном, режиме создания ключа и режиме криптографического преобразования данных. Режим и

алгоритм задаются аргументами командной строки, синтаксис которых соответствует принятому в GNU: у каждой опции есть короткое имя из одного дефиса и буквы и длинное имя из двух дефисов и слова. Набор аргументов проверяется до начала работы, при недопустимом наборе печатается сообщение об ошибке; при конфликте опций приоритет отдается выводу справки.

Надежность и защита данных. Исключения не должны покидать границу динамической библиотеки: об ошибке библиотека сообщает ненулевым кодом завершения, а исполняемый модуль перехватывает исключения, печатает сообщение и завершается с ненулевым кодом возврата. Ключи создаются только криптостойким генератором псевдослучайных чисел операционной системы. Буферы, в которых побывал ключ или открытый текст, принудительно перезаписываются перед освобождением, причем перезапись не должна удаляться оптимизатором.

Ресурсы и средства разработки. При обработке файлов размером до 1 ГБ расход оперативной памяти не должен превышать 128 МБ. Язык программирования C++ стандарта 2017 года, процедурный стиль, стандартные контейнеры и стандартные алгоритмы, сборка средствами CMake версии не ниже 3.10, максимальный уровень диагностики компилятора с трактовкой предупреждений как ошибок. Целевая платформа — Linux с ядром версии 5.x и выше, с возможностью переноса под Windows 10 и 11 без изменения архитектуры.

Сопровождение и проверка. В составе проекта собирается отдельный исполняемый файл, предназначенный только для проверки библиотек алгоритмов, не менее трех проверок на каждый алгоритм. Подготавливаются сценарии установки и удаления для Linux и для Windows. Разработка ведется в системе контроля версий Git с отдельной веткой на каждую задачу и без принудительной перезаписи основной ветки. Комплект сдаваемых документов включает техническое задание [1], пояснительную записку [3], текст программы [2] и руководство оператора [4].

Два требования задания выполнены в форме обоснованного отказа. Дополнение по стандарту PKCS#7 предписано для случая реализации блочных шифров, а создание вектора инициализации — для алгоритмов, которые вектор используют. Все три реализованных шифра побайтовые: они не собирают данные в блоки фиксированного размера и не имеют состояния, которое требовалось бы задавать случайным значением в начале работы, поэтому оба требования неприменимы.

### **2.1.1 Постановка задачи**

Необходимо разработать программное средство «Multi-Algo Cryptotool», реализующее шифрование и дешифрование текстовых данных и файлов с использованием динамически подключаемых библиотек криптографических алгоритмов. Программа должна выполнять следующие основные действия:

1. печатать справку о синтаксисе вызова, перечне опций и перечне поддерживаемых алгоритмов;
2. разбирать аргументы командной строки и проверять допустимость их набора до открытия файлов и до загрузки библиотеки;
3. загружать во время выполнения библиотеку выбранного алгоритма и получать из нее имя алгоритма и длину ключа;
4. создавать ключ выбранного алгоритма криптостойким генератором псевдослучайных чисел операционной системы;
5. читать готовый ключ из файла или из стандартного потока ввода и сохранять используемый ключ в файл или в стандартный поток вывода;
6. читать данные из файла или из стандартного потока ввода, шифровать или расшифровывать их выбранным алгоритмом и записывать результат в файл или в стандартный поток вывода;
7. обрабатывать данные кусками постоянного размера, чтобы расход оперативной памяти не зависел от размера обрабатываемого файла;
8. затирать буферы с ключом и открытыми данными перед освобождением памяти;

9. сообщать об ошибке текстом в стандартный поток ошибок и ненулевым кодом возврата.

## **2.2 Характеристика задачи**

2.2.1 Программа «Multi-Algo Cryptotool» предназначена для шифрования и расшифрования данных произвольного типа тремя классическими алгоритмами и для создания ключей к ним. Программа применяется в учебном процессе кафедры защиты информации как наглядное пособие при изучении криптографических алгоритмов и приемов построения программ с модульной архитектурой. Стойкость реализованных шифров недостаточна для защиты информации ограниченного доступа, поэтому применение программы за пределами учебных задач не предполагается.

2.2.2 Программа используется пользователем для получения ключа выбранного алгоритма, для преобразования этим ключом файла или потока данных, для обратного преобразования тем же ключом, а также для включения криптографического преобразования в конвейер вместе с другими программами командной строки.

2.2.3 Периодичность решения задачи по запросу пользователя. Каждый запуск программы решает ровно одну задачу: печатает справку, создает ключ либо преобразует один поток данных от начала до конца.

2.2.4 Прекращение автоматизированного решения задачи происходит при исчерпании входного потока данных: программа сбрасывает выходной поток, проверяет его состояние и завершается с кодом возврата 0. В информационно-справочном режиме работа прекращается сразу после печати справки, в режиме создания ключа — после записи ключа. Досрочное прекращение происходит при любой ошибке: текст ошибки печатается в стандартный поток ошибок в виде `cryptum: текст`, код возврата при этом равен 1.

2.2.5 Связь с другими задачами присутствует. Она имеет три вида. Во-первых, программа читает данные из стандартного потока ввода и записывает

результат в стандартный поток вывода, поэтому включается в конвейер вместе с любой другой программой командной строки: архиватор может подавать программе поток на шифрование, а расшифрованный поток может передаваться обратно архиватору. Во-вторых, задача шифрования связана с задачей расшифрования файлом ключа: ключ, созданный и сохраненный при шифровании, служит входными данными для последующего расшифрования, и без него данные восстановить невозможно. В-третьих, тестовая утилита `test_algorithms` загружает те же библиотеки алгоритмов тем же кодом, что и основная программа, и решает задачу проверки правильности их работы.

2.2.6 Специальных ограничений на временные характеристики решения задачи не налагается. Техническое задание ограничивает только расход вычислительных ресурсов: при обработке файлов размером до 1 ГБ расход оперативной памяти не должен превышать 128 МБ. Время работы определяется скоростью ввода-вывода и объемом данных и специально не нормируется.

2.2.7 Специальные требования на уровень подготовки пользователя налагаются. Графического интерфейса и диалоговых подсказок у программы нет, поэтому пользователь должен уметь работать в командной строке операционной системы: запускать программу с аргументами, перенаправлять потоки ввода и вывода и соединять программы в конвейер. Кроме того, пользователь должен понимать, что файл ключа необходим для расшифрования и что при его утрате зашифрованные данные восстановить невозможно. Специальных знаний в области криптографии для работы с программой не требуется.

## 2.3 Алгоритм решения

Ниже приведен порядок работы программы от запуска до завершения с указанием функций, которые выполняют каждый шаг.

1. Запуск программы. Управление получает функция `main`, размещенная в модуле `src/main.cpp`.

2. Настройка платформозависимых свойств вызовом `initialise_platform()`. Под Windows устанавливается кодовая страница консоли UTF-8 и двоичный режим стандартных потоков; под Linux функция не делает ничего.
3. Разбор командной строки вызовом функции `parse_command_line`, которой передаются число аргументов и массив их строк.
  - 3.1. Если аргументов нет или где-либо встретилась опция `-h` либо `--help`, что проверяет функция `requests_help`, возвращается набор параметров с режимом `MODE_HELP`, и дальнейшие проверки не выполняются.
  - 3.2. Иначе аргументы просматриваются слева направо. Значение опции берет функция `take_value`, которая принимает как отдельное слово, так и запись через знак равенства; опции без аргумента проверяет функция `reject_attached_value`. Неизвестная опция и лишнее позиционное слово приводят к исключению.
  - 3.3. Проверяется наличие обязательных опций `--algorithm` и `--mode`; значение режима превращается в элемент перечисления функцией `parse_mode`.
  - 3.4. Функция `validate` проверяет совместимость опций: применимость опций к выбранному режиму, наличие источника ключа, запрет одновременного чтения данных и ключа со стандартного ввода и запрет одновременной записи данных и ключа в стандартный вывод.
  - 3.5. Отсутствующим опциям входных и выходных данных присваивается значение `-`, означающее стандартный поток.
4. Путь справки. Если выбран режим `MODE_HELP`, функция `print_help` печатает справку, перечень поддерживаемых алгоритмов для которой дает функция `supported_algorithms`, после чего программа завершается с кодом возврата 0.

5. Иначе выполняется функция `run`.

- 5.1. Имя алгоритма превращается в базовое имя библиотеки функцией `library_base_name`. Имя, отсутствующее в перечне, приводит к исключению.
- 5.2. Библиотека загружается функцией `load_module`. Она получает имя файла библиотеки от `library_file_name`, открывает библиотеку функцией `open_library`, которая под Linux вызывает `dlopen` с флагами `RTLD_NOW` и `RTLD_LOCAL`, и извлекает пять адресов функцией `resolve`.
- 5.3. Запрашиваются метаданные алгоритма вызовом `get_algorithm_info`. Нулевой указатель, пустое имя или нулевая длина ключа приводят к исключению.
- 5.4. Под ключ объявленной длины отводится затираемый буфер `SecureBuffer`.
- 5.5. Ключ либо создается вызовом `run_key_generation`, который обращается к функции библиотеки `generate_key` и проверяет длину созданного ключа, либо читается функцией `read_key`; длина файла ключа проверяется приемом чтения одного лишнего байта.
- 5.6. Если задано сохранение ключа, функция `write_key` записывает ключ и сужает права доступа к файлу до 0600, пока файл еще пуст.
- 5.7. В режиме создания ключа работа на этом заканчивается.
- 5.8. Открываются потоки данных функциями `open_input` и `open_output`; значение - означает стандартный поток.
- 5.9. Выполняется функция `run_transformation`. Она создает входной и выходной затираемые буферы, размер второго определяет функция библиотеки `get_output_size`, и повторяет цикл: `read_bytes` читает очередной кусок размером до 64 КиБ; если прочитано ноль байт, цикл

прекращается; иначе вызывается функция библиотеки `encrypt` или `decrypt`, которой передаются ключ, кусок данных, выходной буфер и абсолютное смещение куска в потоке; код завершения проверяет функция `check_status`; `write_bytes` выводит записанные библиотекой байты; смещение увеличивается на число прочитанных байт.

5.10. Выходной поток сбрасывается, его состояние проверяется еще раз.

6. Завершение работы. Деструктор `SecureBuffer` затирает каждый буфер, в котором побывали ключ или данные, деструктор `CryptoModule` закрывает библиотеку вызовом `dlclose`, функция `main` возвращает 0.
7. Обработка ошибок. Любое исключение, возникшее на шагах 2–5, перехватывается единственным блоком `catch` в функции `main`. Текст ошибки печатается в стандартный поток ошибок в виде `cryptum: текст`, код возврата равен 1.

## 2.4 Руководство пользователя

### 2.4.1 Введение

2.4.1.1 Программа «Multi-Algo Cryptotool» предназначена для шифрования и расшифрования данных произвольного типа одним из трех классических алгоритмов и для создания ключей к этим алгоритмам. Исполняемый файл носит имя `cryptum`, под этим же именем программа вызывается из командной строки.

2.4.1.2 Программа предоставляет пользователю следующие возможности: получить справку о синтаксисе вызова и перечне поддерживаемых алгоритмов; создать ключ выбранного алгоритма и сохранить его в файл или вывести в стандартный поток вывода; зашифровать файл или поток данных готовым либо созданным на месте ключом; расшифровать файл или поток данных готовым ключом; включить программу в конвейер

вместе с другими программами командной строки; подать ключ из файла, из стандартного потока ввода или из подстановки процесса.

2.4.1.3 Программа реализована на алгоритмическом языке C++ стандарта 2017 года [12]. Работает в любой среде совместимой с Linux. Дискковой памяти для запуска программы требуется не менее 64 МБ. Оперативной памяти для нормальной работы программы требуется не менее 512 МБ.

## **2.4.2 Описание операций**

Основные функции программы «Multi-Algo Cryptotool» включают вывод справки, создание ключа, шифрование данных, расшифрование данных и работу в составе конвейера. Далее описаны все функции системы, а также формы и диалоги для ввода и вывода информации.

Диалоговых форм и экранных меню у программы нет. Все управление задается аргументами командной строки и проверяется до начала работы, данные и ключ поступают из файлов или из стандартных потоков, а результат уходит в файл или в стандартный поток вывода. Поэтому каждая операция показана ниже как сеанс работы в окне терминала: набранная команда и ответ программы.

Вывод справки. Справка печатается, если аргументы отсутствуют или если где-либо в командной строке встретилась опция `-h` либо `--help`. Справка имеет приоритет над любой другой опцией и над любой ошибкой в наборе опций, код возврата при этом нулевой (рисунок 2.1).

```

$ cryptum --help
Usage: cryptum --algorithm NAME --mode MODE [options]

Options:
  -h, --help                print this help and exit
  -a, --algorithm NAME      algorithm to use
  -m, --mode MODE           encrypt, decrypt or generate-key
  -k, --key PATH            read the key from PATH, '-' for standard input
  -i, --input PATH          read the data from PATH, '-' or omitted for standard input
  -o, --output PATH         write the data to PATH, '-' or omitted for standard output
  -g, --generate-key        generate a fresh key and use it for this operation
  -s, --save-key PATH       write the key in use to PATH, '-' for standard output
  -w, --write-key           equivalent to --save-key -

An option takes its argument as '--option value', '--option=value' or
'-o value'; combined short options and attached short values are not supported.

Algorithms:
  substitution, beaufort, trithemius

```

Рисунок 2.1 — Вывод справки о программе

Создание ключа. Режим выбирается значением `generate-key` опции `--mode`. Программа создает ключ выбранного алгоритма криптостойким генератором псевдослучайных чисел операционной системы и записывает его туда, куда указывает опция `--save-key` или `--write-key`. Ключ шифра Бофора занимает 16 байт; файл ключа создается с правами доступа 0600 (рисунок 2.2).

```

$ cryptum --algorithm beaufort --mode generate-key --save-key key.bin
$ wc -c < key.bin
16
$ ls -l key.bin | cut -c1-10
-rw-----
$ xxd -p key.bin
9a786b96a316763051055c7e63a682da

```

Рисунок 2.2 — Создание ключа шифра Бофора

Шифрование. Режим выбирается значением `encrypt` опции `--mode`. Источник данных задается опцией `--input`, приемник результата опцией `--output`, готовый ключ опцией `--key`. Длина результата равна длине исходных данных (рисунок 2.3).

```
$ cat data.txt
Отчет по расчетно-графической работе.
$ wc -c < data.txt
70
$ cryptum -a beaufort -m encrypt -k key.bin -i data.txt -o data.enc
$ xxd -p data.enc
cada9a14d28fa67b80833caea4d6c4bac9f89be6d295a5a981508bfc93e9
b21c6da8b8c52346c65fcd35a4addcd6cd0919a8b1c6e546bd1080858cce
93f5b21cc9f69be1750c
```

Рисунок 2.3 — Шифрование файла готовым ключом

Расшифрование. Режим выбирается значением `decrypt` опции `--mode`. Расшифрование тем же ключом восстанавливает исходные данные побайтово, что подтверждает сравнение файлов (рисунок 2.4).

```
$ cryptum -a beaufort -m decrypt -k key.bin -i data.enc -o out.txt
$ cmp data.txt out.txt && echo identical
identical
$ cat out.txt
Отчет по расчетно-графической работе.
```

Рисунок 2.4 — Расшифрование файла и сравнение с исходным

Работа в конвейере. Если опция входных данных не указана, данные читаются со стандартного потока ввода; если не указана опция выходных данных, результат идет в стандартный поток вывода. В примере на рисунке 2.5 данные поступают по конвейеру, ключ простой замены создается на месте опцией `--generate-key` и сохраняется в файл опцией `--save-key`, после чего тот же ключ используется для обратного преобразования. Ключ простой замены занимает 256 байт.

```
$ cat data.txt | cryptum -a substitution -m encrypt -g -s tab.key > data.sub
$ wc -c < tab.key
256
$ cat data.sub | cryptum -a substitution -m decrypt -k tab.key
Отчет по расчетно-графической работе.
```

Рисунок 2.5 — Работа программы в составе конвейера

Сообщение об ошибке. Текст ошибки печатается в стандартный поток ошибок в виде `cryptum: текст`, код возврата при этом равен 1. В стандартный поток вывода диагностика не попадает никогда. На рисунке 2.6 показаны две ошибки: отсутствующий входной файл и недопустимый набор опций.

```
$ cryptum -a beaufort -m encrypt -k key.bin -i nosuch.txt -o out.enc
cryptum: cannot open input file 'nosuch.txt'
$ echo $?
1
$ cryptum -a beaufort -m decrypt -i data.enc
cryptum: encryption and decryption require --key or --generate-key
```

Рисунок 2.6 — Сообщения об ошибках и код возврата

### 2.4.3 Сообщения пользователю

При работе с программой могут появиться следующие сообщения, представленные в таблице 2.1. Сообщения выдаются на английском языке в том виде, в каком их печатает программа. Прописными латинскими буквами обозначены места, в которые программа подставляет фактическое значение: `OPTION` — имя опции вместе с дефисами, `PATH` — путь к файлу, `WORD` — слово командной строки, `NAME` — имя алгоритма или режима, `N` — число байт, `OPERATION` — название операции, принимающее значения `key generation`, `encryption` или `decryption`.

Таблица 2.1 — Сообщения пользователю

| Текст сообщения программы                          | Ответ пользователя                                                                                |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>option OPTION requires an argument</code>    | Указать значение опции                                                                            |
| <code>option OPTION takes no argument</code>       | Убрать знак равенства и значение у опции <code>--generate-key</code> или <code>--write-key</code> |
| <code>unknown option 'OPTION'</code>               | Сверить имя опции со справкой, которую печатает опция <code>--help</code>                         |
| <code>unexpected positional argument 'WORD'</code> | Убрать лишнее слово либо поставить перед ним нужную опцию                                         |
| <code>missing required option --algorithm</code>   | Добавить опцию <code>--algorithm</code> с именем алгоритма                                        |

| Текст сообщения программы                                                                            | Ответ пользователя                                                                                       |
|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| missing required option --mode                                                                       | Добавить опцию --mode со значением encrypt, decrypt или generate-key                                     |
| unknown mode 'NAME'; expected encrypt, decrypt or generate-key                                       | Указать одно из трех допустимых значений режима                                                          |
| unknown algorithm 'NAME'; supported algorithms are substitution, beaufort, trithemius                | Указать одно из трех имен алгоритмов, перечисленных в сообщении                                          |
| --input is not used in generate-key mode                                                             | Убрать опцию --input                                                                                     |
| --output is not used in generate-key mode                                                            | Убрать опцию --output; ключ сохраняется опцией --save-key                                                |
| --key is not used in generate-key mode                                                               | Убрать опцию --key                                                                                       |
| --generate-key is not used in generate-key mode                                                      | Убрать опцию --generate-key                                                                              |
| generate-key mode requires --save-key or --write-key                                                 | Добавить опцию --save-key с именем файла или опцию --write-key                                           |
| --key and --generate-key cannot be combined                                                          | Оставить только одну из двух опций                                                                       |
| encryption and decryption require --key or --generate-key                                            | Добавить опцию --key с именем файла ключа либо, при шифровании, опцию --generate-key                     |
| --generate-key cannot be used in decrypt mode                                                        | Указать опцию --key с тем ключом, которым данные были зашифрованы                                        |
| --save-key and --write-key cannot be used in decrypt mode                                            | Убрать опцию --save-key или --write-key                                                                  |
| --generate-key requires --save-key or --write-key, otherwise the generated key is lost irrecoverably | Добавить опцию --save-key с именем файла или опцию --write-key: без ключа данные расшифровать невозможно |
| the data and the key cannot both be read from standard input                                         | Указать файл в опции --input или в опции --key либо подать ключ через подстановку процесса               |
| the data and the key cannot both be written to standard output                                       | Указать файл в опции --output или в опции --save-key                                                     |
| cannot open input file 'PATH'                                                                        | Проверить путь к файлу и права доступа к нему                                                            |
| cannot open output file 'PATH'                                                                       | Проверить путь, наличие каталога и права на запись                                                       |
| the key for NAME must be exactly N bytes long                                                        | Взять файл ключа, созданный для этого же алгоритма, либо создать новый ключ                              |
| reading the input failed                                                                             | Проверить исправность носителя и повторить операцию                                                      |

| Текст сообщения программы                                 | Ответ пользователя                                                                                                                       |
|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| writing the output failed                                 | Освободить место на диске, проверить получателя конвейера и повторить операцию                                                           |
| writing the key failed                                    | Освободить место на диске и повторить операцию                                                                                           |
| OPERATION failed: the key is not valid for this algorithm | Создать ключ средствами самой программы в режиме generate-key: для простой замены ключ обязан быть перестановкой всех 256 значений байта |
| OPERATION failed: the secure random source is unavailable | Проверить состояние операционной системы и повторить операцию                                                                            |

#### 2.4.4 Аварийные ситуации

К аварийным ситуациям относятся: отсутствие библиотеки выбранного алгоритма рядом с исполняемым файлом; недоступность генератора псевдослучайных чисел операционной системы; исчерпание свободного места на диске или разрыв конвейера при записи результата; отказ носителя при чтении входных данных; утрата файла ключа, после которой зашифрованные данные восстановить невозможно. Во всех перечисленных случаях программа печатает в стандартный поток ошибок сообщение из таблицы 2.1 или 2.2 и завершается с кодом возврата 1, не оставляя после себя незакрытых файлов и не затирая исходные данные: входной файл программа только читает.

Если исполняемый модуль программы не запускается, либо программа сообщает, что библиотека алгоритма не найдена, не экспортирует требуемую функцию или вернула непригодные метаданные, необходимо обратиться к разработчику программы. Такие сообщения означают несовпадение версий исполняемого файла и библиотек и устраняются повторной сборкой или переустановкой программы.

#### 2.5 Руководство системного программиста

Программа реализована на языке C++ стандарта 2017 года [12]. Она состоит из нескольких модулей, каждый из которых выполняет определенный набор функций. Сборка описана единственным файлом CMakeLists.txt в

корневом каталоге проекта и выполняется системой CMake версии не ниже 3.10; сборка дает исполняемый файл `cryptum`, три динамические библиотеки алгоритмов, статическую библиотеку общих модулей и тестовую утилиту `test_algorithms`. Сценарии установки и удаления для Linux написаны на языке командного интерпретатора Bash, сценарии для Windows — на языке PowerShell.

Модули программы:

- `include/crypto_abi.h` — модуль для описания двоичного интерфейса между исполняемым файлом и библиотеками алгоритмов; файла реализации не имеет, включается и исполняемым файлом, и каждой библиотекой;
- `src/main.cpp` — модуль для задания порядка работы, печати справки, перехвата исключений и формирования кода возврата; собственного заголовочного файла не имеет;
- `src/cli_options.h / src/cli_options.cpp` — модуль для разбора набора аргументов командной строки и проверки его допустимости;
- `src/algorithm_registry.h / src/algorithm_registry.cpp` — модуль для хранения перечня поддерживаемых алгоритмов и перехода от имени алгоритма к базовому имени библиотеки;
- `src/crypto_module.h / src/crypto_module.cpp` — модуль для формирования имени файла библиотеки, ее загрузки, получения пяти адресов функций интерфейса и владения дескриптором библиотеки;
- `src/byte_io.h / src/byte_io.cpp` — модуль для двоичного чтения и записи, выбора между файлом и стандартным потоком, чтения и сохранения ключа;
- `src/pipeline.h / src/pipeline.cpp` — модуль для потоковой обработки данных кусками и превращения кодов завершения библиотеки в сообщения об ошибках;

- `src/platform.h / src/platform.cpp` — модуль для настройки кодовой страницы консоли и двоичного режима стандартных потоков под Windows;
- `common/random_bytes.h / common/random_bytes.cpp` — модуль для обращения к криптостойкому генератору псевдослучайных чисел операционной системы;
- `common/secure_memory.h / common/secure_memory.cpp` — модуль для затирания памяти и для типа, владеющего затираемым буфером;
- `lib/substitution/substitution.cpp` — модуль для реализации шифра простой замены, собирается в библиотеку `libsubstitution.so`;
- `lib/beaufort/beaufort.cpp` — модуль для реализации шифра Бофора, собирается в библиотеку `libbeaufort.so`;
- `lib/trithemius/trithemius.cpp` — модуль для реализации шифра Тритемия, собирается в библиотеку `libtrithemius.so`;
- `tests/test_algorithms.cpp` — модуль для проверки библиотек алгоритмов, собирается в отдельный исполняемый файл `test_algorithms`.

Функции внутри модулей. Каждая библиотека алгоритма экспортирует ровно пять функций, объявленных в `include/crypto_abi.h` в блоке `extern "C"`:

- `const AlgorithmInfo* get_algorithm_info(void)` — функция для получения метаданных алгоритма: имени алгоритма и длины ключа;
- `size_t get_output_size(size_t input_size, int operation_type)` — функция для вычисления размера выходного буфера по размеру входных данных и виду операции;
- `int generate_key(MutBuffer* key)` — функция для создания ключа в переданном буфере;

- `int encrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t stream_offset)` — функция для шифрования одного куска данных с учетом его абсолютного смещения в потоке;
- `int decrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t stream_offset)` — функция для расшифрования одного куска данных с учетом его абсолютного смещения в потоке.

Функции исполняемого файла и общих модулей перечислены ниже; все они, кроме функций модуля `main`, объявлены в пространстве имен `cryptum`:

- `void print_help()` — функция для печати справки о синтаксисе вызова, перечне опций и перечне алгоритмов;
- `void run(const cryptum::CliOptions& options)` — функция для выполнения основной работы после разбора аргументов;
- `int main(int argument_count, char** arguments)` — функция для запуска программы, перехвата исключений и формирования кода возврата;
- `CliOptions parse_command_line(int argument_count, char** arguments)` — функция для разбора аргументов командной строки и проверки их набора;
- `std::string supported_algorithms()` — функция для получения перечня имен поддерживаемых алгоритмов одной строкой;
- `std::string library_base_name(const std::string& algorithm)` — функция для перехода от имени алгоритма к базовому имени его библиотеки;
- `void load_module(CryptoModule& module, const std::string& library_base_name)` — функция для загрузки библиотеки и получения адресов пяти функций интерфейса;
- `std::istream& open_input(const std::string& path, std::ifstream& file)` — функция для открытия источника данных: файла или стандартного потока ввода;
- `std::ostream& open_output(const std::string& path, std::ofstream& file)` — функция для открытия приемника данных: файла или стандартного потока вывода;

- `std::size_t read_bytes(std::istream& stream, std::uint8_t* data, std::size_t size)` — функция для чтения очередного куска данных в двоичном виде;
- `void write_bytes(std::ostream& stream, const std::uint8_t* data, std::size_t size)` — функция для записи куска данных в двоичном виде;
- `void read_key(const std::string& path, const std::string& algorithm_name, std::uint8_t* data, std::size_t size)` — функция для чтения ключа и проверки его длины;
- `void write_key(const std::string& path, const std::uint8_t* data, std::size_t size)` — функция для сохранения ключа с правами доступа 0600;
- `void run_key_generation(GenerateKeyFunction generate, SecureBuffer& key)` — функция для создания ключа библиотекой и проверки его длины;
- `void run_transformation(TransformFunction transform, OutputSizeFunction output_size, OperationType operation, ConstBuffer key, std::istream& input, std::ostream& output)` — функция для потоковой обработки данных кусками по 64 КиБ;
- `void initialise_platform()` — функция для настройки консоли и стандартных потоков под Windows;
- `void fill_random_bytes(std::uint8_t* data, std::size_t size)` — функция для получения случайных байт у операционной системы;
- `void secure_zero(std::uint8_t* data, std::size_t size)` — функция для затирания буфера, не удаляемого оптимизатором компилятора.

Кроме функций, в исполняемом файле объявлены два типа с деструкторами: `cryptum::SecureBuffer` из модуля `secure_memory` владеет буфером с ключом или открытыми данными и затирает его при разрушении, а `cryptum::CryptoModule` из модуля `crypto_module` владеет дескриптором динамической библиотеки и закрывает его при разрушении. Оба типа обеспечивают освобождение ресурса на любом пути выхода, включая выход по исключению.

Прикладная логика написана в процедурном стиле, полиморфизм в ней не применяется. Обработка данных в контейнерах выполняется стандартными алгоритмами `std::transform`, `std::for_each`, `std::all_of`, `std::iota` и `std::shuffle`; описание их поведения и требований к аргументам взято из справочника [13].

Связь модулей программы между собой представлена ниже (рисунок 2.7).

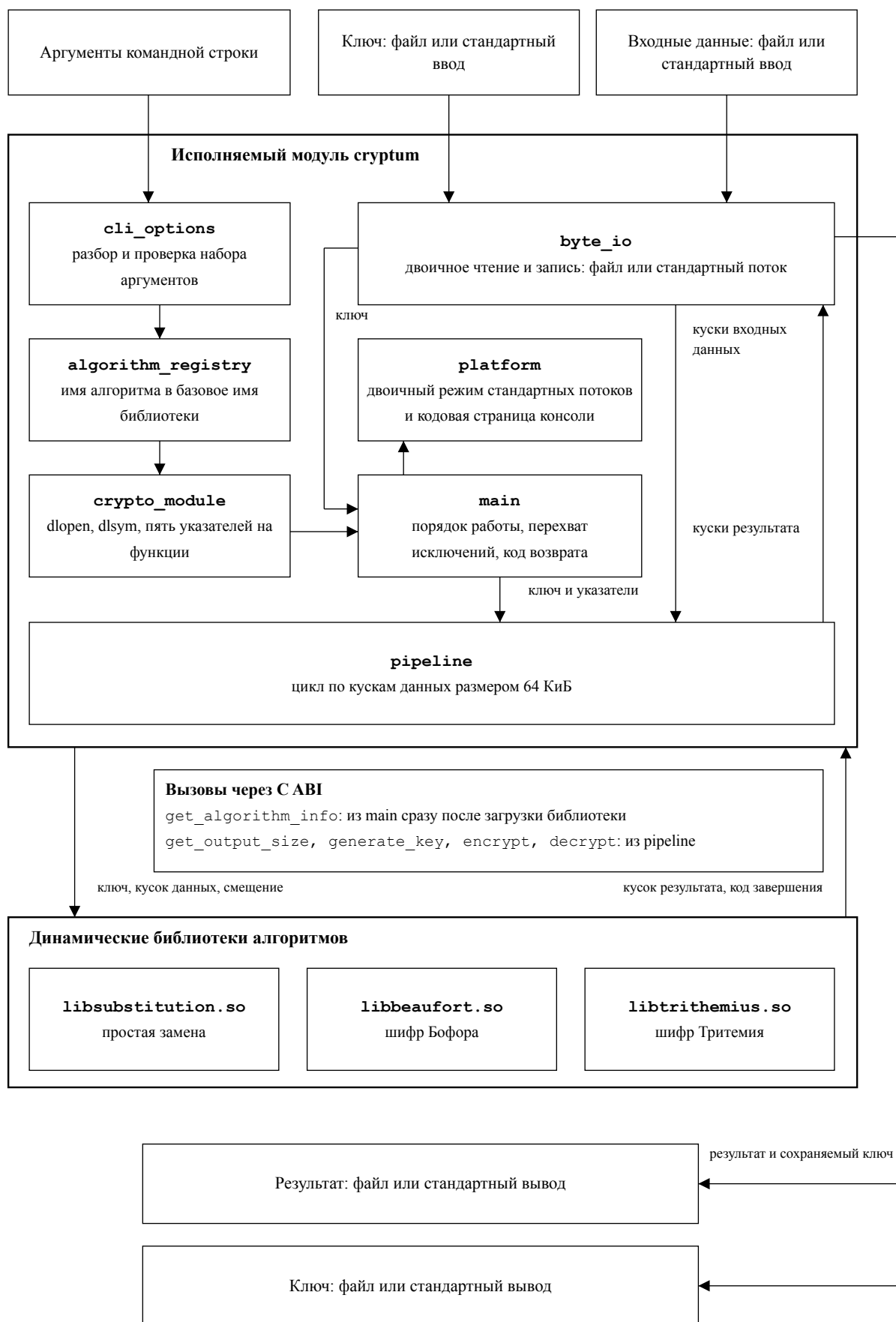


Рисунок 2.7 — Связь модулей программы

Сообщения системному программисту приведены в таблице 2.2. Эти сообщения означают несовпадение версий исполняемого файла и библиотек, отсутствие собранных файлов или ошибку установки и устраняются пересборкой либо переустановкой программы. Прописными латинскими буквами обозначены места, в которые подставляется фактическое значение: FILE — имя файла библиотеки, SYMBOL — имя функции интерфейса, OPERATION — название операции, TEST — имя проверки, TEXT — причина отказа, N и M — числа, DIR — каталог.

Таблица 2.2 — Сообщения системному программисту

| Текст сообщения программы                                                              | Ответ пользователя                                                                                                      |
|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| cannot load the algorithm library<br>'FILE'                                            | Собрать проект заново или переустановить программу, чтобы библиотеки оказались в одном каталоге с исполняемым файлом    |
| the algorithm library does not export<br>'SYMBOL'                                      | Заменить библиотеку на собранную вместе с этой версией программы                                                        |
| the algorithm library reports no<br>usable metadata                                    | Заменить библиотеку на собранную вместе с этой версией программы                                                        |
| key generation failed: the algorithm<br>produced a key of unexpected length            | Заменить библиотеку на собранную вместе с этой версией программы                                                        |
| OPERATION failed: the buffer passed<br>to the algorithm is too small                   | Собрать проект заново целиком: размер выходного буфера, который сообщает библиотека, не согласован с исполняемым файлом |
| OPERATION failed: the algorithm<br>library reported an internal error                  | Освободить оперативную память и повторить операцию                                                                      |
| FAIL TEST: TEXT                                                                        | Собрать проект заново и повторить запуск тестовой утилиты; при повторении отказа обратиться к разработчику              |
| N of M tests passed                                                                    | Убедиться, что N совпадает с M; иначе собрать проект заново                                                             |
| install.sh: administrative rights are<br>required                                      | Повторить запуск сценария установки через sudo                                                                          |
| install.sh: 'DIR/FILE' not found,<br>build the project first                           | Собрать проект или указать каталог с собранными файлами первым аргументом сценария                                      |
| cryptum installed in<br>/usr/local/lib/cryptum and linked as<br>/usr/local/bin/cryptum | Действий не требуется: установка завершена                                                                              |

| Текст сообщения программы                        | Ответ пользователя                            |
|--------------------------------------------------|-----------------------------------------------|
| uninstall.sh: administrative rights are required | Повторить запуск сценария удаления через sudo |
| cryptum removed                                  | Действий не требуется: удаление завершено     |

## 2.6 Контрольный пример

Продemonстрируем работу трех реализованных шифров в программе. Контрольной строкой служит `Nikulin_AB-522_!))`: фамилия исполнителя, номер учебной группы и три знака препинания. Строка записана символами ASCII и занимает ровно 18 байт. Запустим программу и выберем операцию «создание ключа», указав режим `generate-key`: ключ каждого шифра создается генератором псевдослучайных чисел операционной системы и записывается в отдельный файл (рисунок 2.8).

```
$ printf '%s' 'Nikulin_AB-522_!))' > message.txt
$ cat message.txt; echo
Nikulin_AB-522_!))
$ wc -c < message.txt
18
$ xxd -p message.txt
4e696b756c696e5f41422d3532325f212929
$ cryptum -a substitution -m generate-key -s sub.key
$ cryptum -a beaufort -m generate-key -s bft.key
$ cryptum -a trithemius -m generate-key -s trt.key
```

Рисунок 2.8 — Подготовка контрольной строки и создание трех ключей

Зашифруем контрольную строку каждым из трех шифров, затем расшифруем полученные файлы теми же ключами и сравним результат с исходной строкой. Побайтовое совпадение подтверждает команда `cmp`, которая при различии файлов напечатала бы номер первого несовпавшего байта (рисунок 2.9).

```

$ cryptum -a substitution -m encrypt -k sub.key -i message.txt -o message.sub
$ cryptum -a beaufort -m encrypt -k bft.key -i message.txt -o message.bft
$ cryptum -a trithemius -m encrypt -k trt.key -i message.txt -o message.trt
$ cryptum -a substitution -m decrypt -k sub.key -i message.sub -o back.sub
$ cryptum -a beaufort -m decrypt -k bft.key -i message.bft -o back.bft
$ cryptum -a trithemius -m decrypt -k trt.key -i message.trt -o back.trt
$ cmp message.txt back.sub && echo substitution identical
substitution identical
$ cmp message.txt back.bft && echo beaufort identical
beaufort identical
$ cmp message.txt back.trt && echo trithemius identical
trithemius identical

```

Рисунок 2.9 — Шифрование контрольной строки и обратная проверка

Ниже для каждого шифра приведен ручной расчет шифротекста по формулам главы 1 и сравнение его с тем, что напечатала программа. Все значения байт записаны в шестнадцатеричном виде, как их печатает команда `xxd`.

### 2.6.1 Ручной расчёт шифра простой замены

Проведем ручной расчет шифра простой замены и сравним результаты с программой.

Ключом служит таблица замены из 256 байт. Контрольная строка использует только 14 различных значений байта, поэтому из таблицы нужны только 14 ее элементов. Эти элементы извлечены из файла ключа командой, показанной на рисунке 2.10 вместе с результатом шифрования, и выписаны в таблице 2.3.

```

$ wc -c < sub.key
256
$ for b in 21 29 2d 32 35 41 42 4e 5f 69 6b 6c 6e 75; do
>   printf '%s:%s ' $b $(xxd -p -s 0x$b -l 1 sub.key)
> done; echo
21:de 29:38 2d:3d 32:e0 35:e5 41:f3 42:95 4e:53 5f:ea 69:14 6b:54 6c:30 6e:ee 75:ed
$ xxd -p message.sub
531454ed3014eeeeaf3953de5e0e0eade3838

```

Рисунок 2.10 — Ключ простой замены и шифротекст контрольной строки

Таблица 2.3 — Элементы таблицы замены, используемые контрольной строкой

| Символ | Значение байта, hex | Элемент таблицы замены, hex |
|--------|---------------------|-----------------------------|
| !      | 21                  | de                          |
| )      | 29                  | 38                          |
| –      | 2d                  | 3d                          |
| 2      | 32                  | e0                          |
| 5      | 35                  | e5                          |
| A      | 41                  | f3                          |
| B      | 42                  | 95                          |
| N      | 4e                  | 53                          |
| _      | 5f                  | ea                          |
| i      | 69                  | 14                          |
| k      | 6b                  | 54                          |
| l      | 6c                  | 30                          |
| n      | 6e                  | ee                          |
| u      | 75                  | ed                          |

Разберем первые два байта подробно. Байт с номером 0 — это символ N, значение которого равно 4e. По формуле (1) результат равен элементу таблицы замены с номером 4e, то есть  $C[0] = K[4e] = 53$ . Байт с номером 1 — это символ i, значение которого равно 69; элемент таблицы с номером 69 равен 14, поэтому  $C[1] = K[69] = 14$ . Положение байта в потоке на результат не влияет: одинаковые значения байта всегда дают одинаковый результат, что видно по байтам с номерами 1 и 5, 12 и 13, 16 и 17. Остальные байты обработаны так же; расчет приведен в таблице 2.4.

Таблица 2.4 — Ручной расчет шифра простой замены

| Номер байта i | Символ | P[i], hex | Элемент таблицы K[P[i]], hex | C[i], hex |
|---------------|--------|-----------|------------------------------|-----------|
| 0             | N      | 4e        | 53                           | 53        |
| 1             | i      | 69        | 14                           | 14        |
| 2             | k      | 6b        | 54                           | 54        |
| 3             | u      | 75        | ed                           | ed        |
| 4             | l      | 6c        | 30                           | 30        |

| Номер байта i | Символ | P[i], hex | Элемент таблицы K[P[i]], hex | C[i], hex |
|---------------|--------|-----------|------------------------------|-----------|
| 5             | i      | 69        | 14                           | 14        |
| 6             | n      | 6e        | ee                           | ee        |
| 7             | —      | 5f        | ea                           | ea        |
| 8             | A      | 41        | f3                           | f3        |
| 9             | B      | 42        | 95                           | 95        |
| 10            | –      | 2d        | 3d                           | 3d        |
| 11            | 5      | 35        | e5                           | e5        |
| 12            | 2      | 32        | e0                           | e0        |
| 13            | 2      | 32        | e0                           | e0        |
| 14            | —      | 5f        | ea                           | ea        |
| 15            | !      | 21        | de                           | de        |
| 16            | )      | 29        | 38                           | 38        |
| 17            | )      | 29        | 38                           | 38        |

Выписанные подряд значения последнего столбца таблицы 2.4 дают 531454ed3014eeeeaf3953de5e0e0eade3838. Программа напечатала ту же последовательность байт (рисунок 2.10). Результаты совпали с результатами проверки ручным расчетом, программа работает корректно.

### 2.6.2 Ручной расчёт шифра Бофора

Проведем ручной расчет шифра Бофора и сравним результаты с программой.

Ключ шифра Бофора занимает 16 байт и в шестнадцатеричной записи равен e6ef394782caacfdbd2d4bda04358777. Ключ и результат шифрования контрольной строки напечатаны программой (рисунок 2.11).

```
$ xxd -p bft.key
e6ef394782caacfdbd2d4bda04358777
$ xxd -p message.bft
9886ced216613e9e7ceb1ea5d2032856bdc6
```

Рисунок 2.11 — Ключ шифра Бофора и шифротекст контрольной строки

Разберем первые два байта подробно. Байту с номером 0 соответствует байт ключа с номером  $0 \bmod 16 = 0$ , то есть  $K[0] = e6$ . По формуле (3) результат

равен  $C[0] = (K[0] - P[0]) \bmod 256 = (e6 - 4e) \bmod 256$ ; в десятичной записи это  $230 - 78 = 152$ , что дает 98. Байту с номером 1 соответствует байт ключа с номером 1, то есть  $K[1] = ef$ ; результат равен  $C[1] = (ef - 69) \bmod 256$ , в десятичной записи  $239 - 105 = 134$ , что дает 86. Начиная с байта с номером 16 ключ повторяется с начала: номер байта ключа снова равен нулю. Расчет всех байт приведен в таблице 2.5.

Таблица 2.5 — Ручной расчет шифра Бофора

| Номер байта $i$ | Символ | $P[i]$ , hex | $i \bmod 16$ | $K[i \bmod 16]$ , hex | $C[i]$ , hex |
|-----------------|--------|--------------|--------------|-----------------------|--------------|
| 0               | N      | 4e           | 0            | e6                    | 98           |
| 1               | i      | 69           | 1            | ef                    | 86           |
| 2               | k      | 6b           | 2            | 39                    | ce           |
| 3               | u      | 75           | 3            | 47                    | d2           |
| 4               | l      | 6c           | 4            | 82                    | 16           |
| 5               | i      | 69           | 5            | ca                    | 61           |
| 6               | n      | 6e           | 6            | ac                    | 3e           |
| 7               | —      | 5f           | 7            | fd                    | 9e           |
| 8               | A      | 41           | 8            | bd                    | 7c           |
| 9               | B      | 42           | 9            | 2d                    | eb           |
| 10              | -      | 2d           | 10           | 4b                    | 1e           |
| 11              | 5      | 35           | 11           | da                    | a5           |
| 12              | 2      | 32           | 12           | 04                    | d2           |
| 13              | 2      | 32           | 13           | 35                    | 03           |
| 14              | —      | 5f           | 14           | 87                    | 28           |
| 15              | !      | 21           | 15           | 77                    | 56           |
| 16              | )      | 29           | 0            | e6                    | bd           |
| 17              | )      | 29           | 1            | ef                    | c6           |

Выписанные подряд значения последнего столбца таблицы 2.5 дают 9886ced216613e9e7ceb1ea5d2032856bdc6. Программа напечатала ту же последовательность байт (рисунок 2.11). Результаты совпали с результатами проверки ручным расчетом, программа работает корректно.

### 2.6.3 Ручной расчёт шифра Тритемия

Проведем ручной расчет шифра Тритемия и сравним результаты с программой.

Ключ шифра Тритемия занимает 16 байт и в шестнадцатеричной записи равен 89e709428b75fd8c1fec279bd0ad56fb. Ключ и результат шифрования контрольной строки напечатаны программой (рисунок 2.12).

```
$ xxd -p trt.key
89e709428b75fd8c1fec279bd0ad56fb
$ xxd -p message.trt
d75176bafbe371f268375edb0eccc32bc221
```

Рисунок 2.12 — Ключ шифра Тритемия и шифротекст контрольной строки

Разберем первые два байта подробно. Байту с номером 0 соответствует байт ключа с номером  $0 \bmod 16 = 0$ , то есть  $K[0] = 89$ ; слагаемое, задающее нарастающий сдвиг, равно самому номеру байта, то есть 00. По формуле (5) результат равен  $C[0] = (P[0] + K[0] + 0) \bmod 256 = (4e + 89 + 00) \bmod 256$ ; в десятичной записи это  $78 + 137 + 0 = 215$ , что дает d7. Для байта с номером 1 получаем  $K[1] = e7$  и слагаемое 01, поэтому  $C[1] = (69 + e7 + 01) \bmod 256$ ; в десятичной записи  $105 + 231 + 1 = 337$ , а  $337 - 256 = 81$ , что дает 51. Уже на втором байте сумма вышла за границу байта, и приведение по модулю 256 отбросило перенос. Расчет всех байт приведен в таблице 2.6.

Таблица 2.6 — Ручной расчет шифра Тритемия

| Номер байта $i$ | Символ | $P[i]$ , hex | $i \bmod 16$ | $K[i \bmod 16]$ , hex | $i$ , hex | $C[i]$ , hex |
|-----------------|--------|--------------|--------------|-----------------------|-----------|--------------|
| 0               | N      | 4e           | 0            | 89                    | 00        | d7           |
| 1               | i      | 69           | 1            | e7                    | 01        | 51           |
| 2               | k      | 6b           | 2            | 09                    | 02        | 76           |
| 3               | u      | 75           | 3            | 42                    | 03        | ba           |
| 4               | l      | 6c           | 4            | 8b                    | 04        | fb           |
| 5               | i      | 69           | 5            | 75                    | 05        | e3           |
| 6               | n      | 6e           | 6            | fd                    | 06        | 71           |
| 7               | —      | 5f           | 7            | 8c                    | 07        | f2           |

| Номер байта $i$ | Символ | $P[i]$ , hex | $i \bmod 16$ | $K[i \bmod 16]$ , hex | $i$ , hex | $C[i]$ , hex |
|-----------------|--------|--------------|--------------|-----------------------|-----------|--------------|
| 8               | A      | 41           | 8            | 1f                    | 08        | 68           |
| 9               | B      | 42           | 9            | ec                    | 09        | 37           |
| 10              | –      | 2d           | 10           | 27                    | 0a        | 5e           |
| 11              | 5      | 35           | 11           | 9b                    | 0b        | db           |
| 12              | 2      | 32           | 12           | d0                    | 0c        | 0e           |
| 13              | 2      | 32           | 13           | ad                    | 0d        | ec           |
| 14              | —      | 5f           | 14           | 56                    | 0e        | c3           |
| 15              | !      | 21           | 15           | fb                    | 0f        | 2b           |
| 16              | )      | 29           | 0            | 89                    | 10        | c2           |
| 17              | )      | 29           | 1            | e7                    | 11        | 21           |

Выписанные подряд значения последнего столбца таблицы 2.6 дают d75176bafbe371f268375edb0eccc32bc221. Программа напечатала ту же последовательность байт (рисунок 2.12). Результаты совпали с результатами проверки ручным расчетом, программа работает корректно.

## ЗАКЛЮЧЕНИЕ

В ходе расчетно-графической работы разработана консольная программа «Multi-Algo Cryptotool», которая шифрует и расшифровывает данные произвольного типа тремя классическими алгоритмами и создает ключи для этих алгоритмов. Программа собрана, установлена и проверена в операционной системе на базе ядра Linux. Цель работы достигнута: исполняемый файл не содержит ни одной реализации шифра, а добавление нового алгоритма сводится к сборке новой библиотеки и к одной строке в перечне алгоритмов исполняемого модуля.

Задачи, названные во введении, решены следующим образом.

Три классических шифра изучены, их происхождение и свойства изложены в главе 1, а побайтовые варианты, реализованные в программе, записаны формулами (1)–(6) с расшифровкой каждого обозначения. Для каждого шифра показано, почему расшифрование обращает шифрование, и отмечено, чем этот шифр вскрывается.

Двоичный интерфейс между исполняемым модулем и библиотекой алгоритма спроектирован и описан заголовочным файлом `include/crypto_abi.h`. Интерфейс состоит из пяти функций; память для обмена выделяет исполняемый модуль, поэтому функции освобождения памяти в интерфейсе нет. Передача абсолютного смещения куска в потоке позволила применять позиционно зависимый шифр к потоку по частям.

Исполняемый модуль реализован и разделен на семь модулей с разбором и проверкой аргументов командной строки, загрузкой библиотеки во время выполнения и потоковой обработкой данных кусками по 64 КиБ. Состав модулей, их функции и связи приведены в 2.5 и на рисунке 2.7, порядок работы — в 2.3.

Три библиотеки шифров реализованы, каждая экспортирует ровно пять функций интерфейса. Ключи создаются только криптостойким генератором псевдослучайных чисел операционной системы, а буферы, в которых побывали ключ или открытые данные, затираются при разрушении на любом пути выхода.

Тестовая утилита написана и содержит 19 проверок: на каждый алгоритм приходится шесть проверок, включая сверку с заранее выписанным по формуле результатом и совпадение обработки одним вызовом с обработкой по кускам разной длины; последняя проверка убеждается, что библиотека простой замены отвергает ключ, не являющийся перестановкой. Все проверки проходят.

Сценарии установки и удаления подготовлены для Linux и для Windows. Сценарии для Linux проверены, сценарии для Windows относятся к заявленной будущей платформе.

Комплект программной документации оформлен: техническое задание по ГОСТ 19.201-78 [1], пояснительная записка по ГОСТ 19.404-79 [3], текст программы по ГОСТ 19.401-78 [2] и руководство оператора по ГОСТ 19.505-79 [4]. Исходный текст программы и документация хранятся в репозитории Git, ссылка на который приведена в приложении Б.

Правильность работы программы подтверждена контрольным примером: для каждого из трех шифров шифротекст контрольной строки вычислен вручную по формулам главы 1 и совпал с тем, что напечатала программа.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Перечень составлен по ГОСТ Р 7.0.5-2008 [5].

1. ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. Введ. 1980-01-01. М.: Стандартиформ, 2010.
2. ГОСТ 19.401-78. Единая система программной документации. Текст программы. Требования к содержанию и оформлению. Введ. 1980-01-01. М.: Стандартиформ, 2010.
3. ГОСТ 19.404-79. Единая система программной документации. Пояснительная записка. Требования к содержанию и оформлению. Введ. 1981-01-01. М.: Стандартиформ, 2010.
4. ГОСТ 19.505-79. Единая система программной документации. Руководство оператора. Требования к содержанию и оформлению. Введ. 1980-01-01. М.: Стандартиформ, 2010.
5. ГОСТ Р 7.0.5-2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления. Введ. 2009-01-01. М.: Стандартиформ, 2008.
6. Алферов А. П., Зубов А. Ю., Кузьмин А. С., Черемушкин А. В. Основы криптографии: учебное пособие. 3-е изд., испр. и доп. М.: Гелиос АРВ, 2005. 480 с.
7. Menezes A. J., van Oorschot P. C., Vanstone S. A. Handbook of Applied Cryptography. Boca Raton: CRC Press, 1996. 780 p.
8. Schneier B. Applied Cryptography: Protocols, Algorithms, and Source Code in C. 2nd ed. New York: John Wiley & Sons, 1996. 758 p.
9. Kahn D. The Codebreakers: The Comprehensive History of Secret Communication from Ancient Times to the Internet. Rev. ed. New York: Scribner, 1996. 1181 p.
10. Жельников В. Криптография от папируса до компьютера. М.: АБФ, 1996. 335 с.

11. Бабаш А. В., Шанкин Г. П. История криптографии. Часть I. М.: Гелиос АРВ, 2002. 240 с.
12. ISO/IEC 14882:2017. Programming languages — C++. Geneva: ISO, 2017.
13. C++ reference [Электронный ресурс]. URL: <https://en.cppreference.com/w/cpp> (дата обращения: 22.09.2026).
14. dlopen(3) [Электронный ресурс] // Linux manual pages. URL: <https://man7.org/linux/man-pages/man3/dlopen.3.html> (дата обращения: 22.09.2026).
15. getrandom(2) [Электронный ресурс] // Linux manual pages. URL: <https://man7.org/linux/man-pages/man2/getrandom.2.html> (дата обращения: 22.09.2026).
16. explicit\_bzero(3) [Электронный ресурс] // Linux manual pages. URL: [https://man7.org/linux/man-pages/man3/explicit\\_bzero.3.html](https://man7.org/linux/man-pages/man3/explicit_bzero.3.html) (дата обращения: 22.09.2026).

# ПРИЛОЖЕНИЕ А

## Отчет антиплагиата

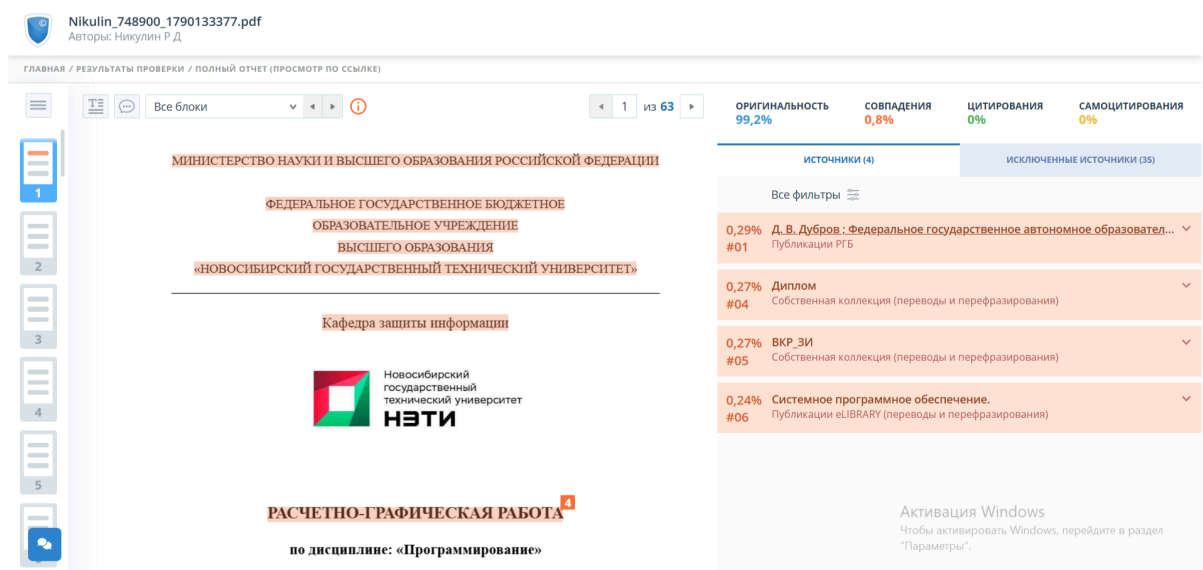


Рисунок А.1 — Отчёт антиплагиата

## **ПРИЛОЖЕНИЕ Б**

Ссылка на Git репозиторий: <https://github.com/PotionsKnight/rgr-cryptum>

## ПРИЛОЖЕНИЕ В

### Текст программы

Ниже приведены все файлы исходного текста, входящие в состав программного изделия: сценарий сборки, заголовочный файл двоичного интерфейса, исходные тексты общих модулей и основного исполняемого модуля, исходные тексты трех динамических библиотек алгоритмов шифрования, исходный текст тестовой утилиты и сценарии установки и удаления для Linux и для Windows. Всего приведено 27 файлов общим объемом 1541 строка. Каждому листингу предшествует путь файла относительно корня репозитория; текст каждого файла воспроизведен без изменений.

#### Листинг В.1 — CMakeLists.txt

```
cmake_minimum_required(VERSION 3.10)
project(cryptum CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)
set(CMAKE_LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)

# Together with the hidden visibility of common this keeps a cipher library exporting
# its five interface functions and nothing else.
set(CMAKE_VISIBILITY_INLINES_HIDDEN ON)

set(CMAKE_BUILD_WITH_INSTALL_RPATH TRUE)
set(CMAKE_INSTALL_RPATH "$ORIGIN")

if(MSVC)
    add_compile_options(/W4 /WX /utf-8)
else()
    add_compile_options(-Wall -Wextra -Wpedantic -Werror)
endif()

add_library(common STATIC common/random_bytes.cpp common/secure_memory.cpp)
target_include_directories(common PUBLIC
    ${CMAKE_CURRENT_SOURCE_DIR}/common
    ${CMAKE_CURRENT_SOURCE_DIR}/include)
set_target_properties(common PROPERTIES
    POSITION_INDEPENDENT_CODE ON
    CXX_VISIBILITY_PRESET hidden)

add_executable(cryptum
    src/main.cpp
    src/cli_options.cpp
    src/byte_io.cpp
    src/platform.cpp
    src/pipeline.cpp
    src/algorithm_registry.cpp
    src/crypto_module.cpp)
target_link_libraries(cryptum PRIVATE common ${CMAKE_DL_LIBS})

add_library(substitution SHARED lib/substitution/substitution.cpp)
target_link_libraries(substitution PRIVATE common)

add_library(beaufort SHARED lib/beaufort/beaufort.cpp)
target_link_libraries(beaufort PRIVATE common)

add_library(trithemius SHARED lib/trithemius/trithemius.cpp)
target_link_libraries(trithemius PRIVATE common)

add_executable(test_algorithms tests/test_algorithms.cpp src/crypto_module.cpp)
target_include_directories(test_algorithms PRIVATE ${CMAKE_CURRENT_SOURCE_DIR}/src)
```

```
target_link_libraries(test_algorithms PRIVATE common ${CMAKE_DL_LIBS})
add_dependencies(test_algorithms substitution beaufort trithemius)
```

## Листинг В.2 — include/crypto\_abi.h

```
#ifndef CRYPTUM_CRYPTO_ABI_H
#define CRYPTUM_CRYPTO_ABI_H

#include <stddef.h>
#include <stdint.h>

extern "C"
{
    struct ConstBuffer
    {
        const uint8_t* data;
        size_t size;
    };

    struct MutBuffer
    {
        uint8_t* data;
        size_t size;
    };

    struct AlgorithmInfo
    {
        const char* algorithm_name;
        size_t key_size;
    };

    enum OperationType
    {
        OPERATION_ENCRYPT = 0,
        OPERATION_DECRYPT = 1
    };

    enum CryptoStatus
    {
        CRYPTO_OK = 0,
        CRYPTO_ERROR_INVALID_KEY = 1,
        CRYPTO_ERROR_INVALID_BUFFER = 2,
        CRYPTO_ERROR_RANDOM_FAILURE = 3,
        CRYPTO_ERROR_INTERNAL = 4
    };

    // stream_offset extends the interface because a position-dependent cipher cannot be applied
    // to a stream chunk by chunk without the absolute index of the first byte of the chunk.
    // generate_key extends it because only the library knows which byte sequences form a valid
    // key for its algorithm, and the executable must make no assumption about that.

    const AlgorithmInfo* get_algorithm_info(void);
    size_t get_output_size(size_t input_size, int operation_type);
    int generate_key(MutBuffer* key);
    int encrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t stream_offset);
    int decrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t stream_offset);
}

#endif
```

## Листинг В.3 — common/random\_bytes.h

```
#ifndef CRYPTUM_RANDOM_BYTES_H
#define CRYPTUM_RANDOM_BYTES_H

#include <cstdint>
#include <cstdlib>

namespace cryptum
{
    void fill_random_bytes(std::uint8_t* data, std::size_t size);
}

#endif
```

## Листинг В.4 — common/random\_bytes.cpp

```
#include "random_bytes.h"
```

```

#include <stdexcept>

#if defined(_WIN32)
# include <bcrypt.h>
# include <windows.h>
#else
# include <cerrno>
# include <sys/random.h>
#endif

namespace cryptum
{

void fill_random_bytes(std::uint8_t* data, std::size_t size)
{
#if defined(_WIN32)
    if (BCryptGenRandom(nullptr, data, static_cast<ULONG>(size), BCRYPT_USE_SYSTEM_PREFERRED_RNG)
        != 0)
        throw std::runtime_error("the secure random source is unavailable");
#else
    std::size_t filled = 0;
    while (filled < size)
    {
        const ssize_t written = getrandom(data + filled, size - filled, 0);
        if (written < 0)
        {
            if (errno == EINTR) continue;
            throw std::runtime_error("the secure random source is unavailable");
        }
        filled += static_cast<std::size_t>(written);
    }
#endif
}

} // namespace cryptum

```

## Листинг В.5 — common/secure\_memory.h

```

#ifndef CRYPTUM_SECURE_MEMORY_H
#define CRYPTUM_SECURE_MEMORY_H

#include <cstdint>
#include <vector>

namespace cryptum
{

void secure_zero(std::uint8_t* data, std::size_t size);

// Owns a buffer that holds key material or plaintext and wipes it on every exit path.
struct SecureBuffer
{
    explicit SecureBuffer(std::size_t size) : bytes(size) {}
    ~SecureBuffer() { secure_zero(bytes.data(), bytes.size()); }

    SecureBuffer(const SecureBuffer&) = delete;
    SecureBuffer& operator=(const SecureBuffer&) = delete;

    std::vector<std::uint8_t> bytes;
};

} // namespace cryptum

#endif

```

## Листинг В.6 — common/secure\_memory.cpp

```

#include "secure_memory.h"

#if defined(_WIN32)
# include <windows.h>
#else
# include <string.h>
#endif

namespace cryptum
{

void secure_zero(std::uint8_t* data, std::size_t size)
{
#if defined(_WIN32)
    SecureZeroMemory(data, size);
#else

```

```

    explicit_bzero(data, size);
#endif
}
} // namespace cryptum

```

## Листинг В.7 — src/algorithm\_registry.h

```

#ifndef CRYPTUM_ALGORITHM_REGISTRY_H
#define CRYPTUM_ALGORITHM_REGISTRY_H

#include <string>

namespace cryptum
{

std::string supported_algorithms();
std::string library_base_name(const std::string& algorithm);
} // namespace cryptum

#endif

```

## Листинг В.8 — src/algorithm\_registry.cpp

```

#include "algorithm_registry.h"

#include <stdexcept>

namespace cryptum
{

namespace
{

struct AlgorithmEntry
{
    const char* algorithm;
    const char* library;
};

const AlgorithmEntry REGISTRY[] = {
    {"substitution", "substitution"},
    {"beaufort", "beaufort"},
    {"trithemius", "trithemius"},
};

} // namespace

std::string supported_algorithms()
{
    std::string names;
    for (const AlgorithmEntry& entry : REGISTRY)
    {
        if (!names.empty()) names += ", ";
        names += entry.algorithm;
    }
    return names;
}

std::string library_base_name(const std::string& algorithm)
{
    for (const AlgorithmEntry& entry : REGISTRY)
        if (algorithm == entry.algorithm) return entry.library;
    throw std::runtime_error("unknown algorithm '" + algorithm + "'; supported algorithms are "
        + supported_algorithms());
}

} // namespace cryptum

```

## Листинг В.9 — src/byte\_io.h

```

#ifndef CRYPTUM_BYTE_IO_H
#define CRYPTUM_BYTE_IO_H

#include <cstdint>
#include <cstdint>
#include <fstream>
#include <istream>
#include <ostream>
#include <string>

namespace cryptum
{

```

```

std::istream& open_input(const std::string& path, std::ifstream& file);
std::ostream& open_output(const std::string& path, std::ofstream& file);

std::size_t read_bytes(std::istream& stream, std::uint8_t* data, std::size_t size);
void write_bytes(std::ostream& stream, const std::uint8_t* data, std::size_t size);
void read_key(const std::string& path, const std::string& algorithm_name, std::uint8_t* data,
              std::size_t size);
void write_key(const std::string& path, const std::uint8_t* data, std::size_t size);
} // namespace cryptum

#endif

```

## Листинг В.10 — src/byte\_io.cpp

```

#include "byte_io.h"

#include <filesystem>
#include <iostream>
#include <stdexcept>
#include <string>

namespace cryptum
{
    std::istream& open_input(const std::string& path, std::ifstream& file)
    {
        if (path == "-") return std::cin;
        file.open(path, std::ios::binary);
        if (!file) throw std::runtime_error("cannot open input file '" + path + "'");
        return file;
    }

    std::ostream& open_output(const std::string& path, std::ofstream& file)
    {
        if (path == "-") return std::cout;
        file.open(path, std::ios::binary | std::ios::trunc);
        if (!file) throw std::runtime_error("cannot open output file '" + path + "'");
        return file;
    }

    std::size_t read_bytes(std::istream& stream, std::uint8_t* data, std::size_t size)
    {
        stream.read(reinterpret_cast<char*>(data), static_cast<std::streamsize>(size));
        if (stream.bad()) throw std::runtime_error("reading the input failed");
        return static_cast<std::size_t>(stream.gcount());
    }

    void write_bytes(std::ostream& stream, const std::uint8_t* data, std::size_t size)
    {
        stream.write(reinterpret_cast<const char*>(data), static_cast<std::streamsize>(size));
        if (!stream) throw std::runtime_error("writing the output failed");
    }

    void read_key(const std::string& path, const std::string& algorithm_name, std::uint8_t* data,
                  std::size_t size)
    {
        std::ifstream file;
        std::istream& stream = open_input(path, file);

        // One byte beyond the expected length is read on purpose: it tells a key file that is too
        // long from one of exactly the right size, without seeking in a stream that may be a pipe.
        std::uint8_t beyond = 0;
        if (read_bytes(stream, data, size) != size || read_bytes(stream, &beyond, 1) != 0)
            throw std::runtime_error("the key for " + algorithm_name + " must be exactly "
                                     + std::to_string(size) + " bytes long");
    }

    void write_key(const std::string& path, const std::uint8_t* data, std::size_t size)
    {
        std::ofstream file;
        std::ostream& stream = open_output(path, file);

        // The permissions are narrowed while the file is still empty, so the key is never
        // readable by anyone else, not even for the time of the write.
        if (file.is_open())
            std::filesystem::permissions(
                path, std::filesystem::perms::owner_read | std::filesystem::perms::owner_write,
                std::filesystem::perm_options::replace);

        write_bytes(stream, data, size);
        stream.flush();
        if (!stream) throw std::runtime_error("writing the key failed");
    }
}

```

```

}
} // namespace cryptum

```

## Листинг В.11 — src/cli\_options.h

```

#ifndef CRYPTUM_CLI_OPTIONS_H
#define CRYPTUM_CLI_OPTIONS_H

#include <string>

namespace cryptum
{
    enum ProgramMode
    {
        MODE_HELP,
        MODE_GENERATE_KEY,
        MODE_ENCRYPT,
        MODE_DECRYPT
    };

    struct CliOptions
    {
        ProgramMode mode = MODE_HELP;
        std::string algorithm;
        std::string key_path;
        std::string input_path;
        std::string output_path;
        std::string save_key_path;
        bool generate_key = false;
        bool has_key = false;
        bool has_input = false;
        bool has_output = false;
        bool has_save_key = false;
    };

    CliOptions parse_command_line(int argument_count, char** arguments);
} // namespace cryptum

#endif

```

## Листинг В.12 — src/cli\_options.cpp

```

#include "cli_options.h"

#include <cstdint>
#include <stdexcept>
#include <vector>

namespace cryptum
{
    namespace
    {
        std::string take_value(const std::string& option, const std::string& attached_value,
                               bool has_attached_value, const std::vector<std::string>& arguments,
                               std::size_t& index)
        {
            if (has_attached_value) return attached_value;
            if (index + 1 >= arguments.size())
                throw std::runtime_error("option " + option + " requires an argument");
            ++index;
            return arguments[index];
        }

        void reject_attached_value(const std::string& option, bool has_attached_value)
        {
            if (has_attached_value) throw std::runtime_error("option " + option + " takes no argument");
        }

        ProgramMode parse_mode(const std::string& name)
        {
            if (name == "encrypt") return MODE_ENCRYPT;
            if (name == "decrypt") return MODE_DECRYPT;
            if (name == "generate-key") return MODE_GENERATE_KEY;
            throw std::runtime_error("unknown mode '" + name
                                     + "'; expected encrypt, decrypt or generate-key");
        }

        bool requests_help(const std::vector<std::string>& arguments)
        {

```

```

    for (const std::string& argument : arguments)
        if (argument == "-h" || argument == "--help") return true;
    return false;
}

void validate(const CliOptions& options)
{
    if (options.mode == MODE_GENERATE_KEY)
    {
        if (options.has_input) throw std::runtime_error("--input is not used in generate-key mode");
        if (options.has_output)
            throw std::runtime_error("--output is not used in generate-key mode");
        if (options.has_key) throw std::runtime_error("--key is not used in generate-key mode");
        if (options.generate_key)
            throw std::runtime_error("--generate-key is not used in generate-key mode");
        if (!options.has_save_key)
            throw std::runtime_error("generate-key mode requires --save-key or --write-key");
        return;
    }

    if (options.has_key && options.generate_key)
        throw std::runtime_error("--key and --generate-key cannot be combined");
    if (!options.has_key && !options.generate_key)
        throw std::runtime_error("encryption and decryption require --key or --generate-key");

    if (options.mode == MODE_DECRYPT)
    {
        if (options.generate_key)
            throw std::runtime_error("--generate-key cannot be used in decrypt mode");
        if (options.has_save_key)
            throw std::runtime_error("--save-key and --write-key cannot be used in decrypt mode");
    }

    if (options.generate_key && !options.has_save_key)
        throw std::runtime_error("--generate-key requires --save-key or --write-key, otherwise the"
                                   " generated key is lost irrecoverably");

    const bool input_from_standard_input = !options.has_input || options.input_path == "-";
    if (input_from_standard_input && options.has_key && options.key_path == "-")
        throw std::runtime_error("the data and the key cannot both be read from standard input");

    const bool output_to_standard_output = !options.has_output || options.output_path == "-";
    if (output_to_standard_output && options.has_save_key && options.save_key_path == "-")
        throw std::runtime_error("the data and the key cannot both be written to standard output");
}
} // namespace

CliOptions parse_command_line(int argument_count, char** arguments)
{
    const std::vector<std::string> given(arguments + 1, arguments + argument_count);

    CliOptions options;
    if (given.empty() || requests_help(given)) return options;

    bool algorithm_given = false;
    bool mode_given = false;
    std::string mode_name;

    for (std::size_t index = 0; index < given.size(); ++index)
    {
        const std::string& argument = given[index];

        std::string name = argument;
        std::string attached_value;
        bool has_attached_value = false;

        const std::size_t separator = argument.find('=');
        if (argument.rfind("--", 0) == 0 && separator != std::string::npos)
        {
            name = argument.substr(0, separator);
            attached_value = argument.substr(separator + 1);
            has_attached_value = true;
        }

        if (name == "-a" || name == "--algorithm")
        {
            options.algorithm = take_value(name, attached_value, has_attached_value, given, index);
            algorithm_given = true;
        }
        else if (name == "-m" || name == "--mode")
        {
            mode_name = take_value(name, attached_value, has_attached_value, given, index);
            mode_given = true;
        }
        else if (name == "-k" || name == "--key")
        {

```

```

        options.key_path = take_value(name, attached_value, has_attached_value, given, index);
        options.has_key = true;
    }
    else if (name == "-i" || name == "--input")
    {
        options.input_path = take_value(name, attached_value, has_attached_value, given, index);
        options.has_input = true;
    }
    else if (name == "-o" || name == "--output")
    {
        options.output_path
            = take_value(name, attached_value, has_attached_value, given, index);
        options.has_output = true;
    }
    else if (name == "-g" || name == "--generate-key")
    {
        reject_attached_value(name, has_attached_value);
        options.generate_key = true;
    }
    else if (name == "-s" || name == "--save-key")
    {
        options.save_key_path
            = take_value(name, attached_value, has_attached_value, given, index);
        options.has_save_key = true;
    }
    else if (name == "-w" || name == "--write-key")
    {
        reject_attached_value(name, has_attached_value);
        options.save_key_path = "-";
        options.has_save_key = true;
    }
    else if (!name.empty() && name[0] == '-')
    {
        throw std::runtime_error("unknown option '" + name + "'");
    }
    else { throw std::runtime_error("unexpected positional argument '" + argument + "'"); }
}

if (!algorithm_given) throw std::runtime_error("missing required option --algorithm");
if (!mode_given) throw std::runtime_error("missing required option --mode");
options.mode = parse_mode(mode_name);
validate(options);

if (options.mode != MODE_GENERATE_KEY)
{
    if (!options.has_input) options.input_path = "-";
    if (!options.has_output) options.output_path = "-";
}

return options;
}
} // namespace cryptum

```

## Листинг В.13 — src/crypto\_module.h

```

#ifndef CRYPTUM_CRYPTOMODULE_H
#define CRYPTUM_CRYPTOMODULE_H

#include "crypto_abi.h"

#include <stddef>
#include <cstdint>
#include <string>

namespace cryptum
{
    // Owns the dynamic library handle so that it is released on every exit path.
    struct CryptoModule
    {
        CryptoModule() = default;
        ~CryptoModule();

        CryptoModule(const CryptoModule&) = delete;
        CryptoModule& operator=(const CryptoModule&) = delete;

        void* handle = nullptr;

        const AlgorithmInfo* (*get_algorithm_info)() = nullptr;
        std::size_t (*get_output_size)(std::size_t, int) = nullptr;
        int (*generate_key)(MutBuffer*) = nullptr;
        int (*encrypt)(ConstBuffer, ConstBuffer, MutBuffer*, std::uint64_t) = nullptr;
        int (*decrypt)(ConstBuffer, ConstBuffer, MutBuffer*, std::uint64_t) = nullptr;
    };
}

```

```

void load_module(CryptoModule& module, const std::string& library_base_name);
} // namespace cryptum

#endif

```

## Листинг В.14 — src/crypto\_module.cpp

```

#include "crypto_module.h"

#include <stdexcept>

#ifdef _WIN32
# include <windows.h>
#else
# include <dlfcn.h>
#endif

namespace cryptum
{
    namespace
    {
        using GenericFunction = void (*)();

        std::string library_file_name(const std::string& base_name)
        {
#ifdef _WIN32
            return base_name + ".dll";
#else
            return "lib" + base_name + ".so";
#endif
        }

        void* open_library(const std::string& file_name)
        {
#ifdef _WIN32
            void* handle = LoadLibraryA(file_name.c_str());
#else
            void* handle = dlopen(file_name.c_str(), RTLD_NOW | RTLD_LOCAL);
#endif
            if (handle == nullptr)
                throw std::runtime_error("cannot load the algorithm library '" + file_name + "'");
            return handle;
        }

        GenericFunction resolve(void* handle, const char* symbol)
        {
#ifdef _WIN32
            GenericFunction function
                = reinterpret_cast<GenericFunction>(GetProcAddress(static_cast<HMODULE>(handle), symbol));
#else
            GenericFunction function = reinterpret_cast<GenericFunction>(dlsym(handle, symbol));
#endif
            if (function == nullptr)
                throw std::runtime_error(std::string("the algorithm library does not export '" + symbol
                    + "'");
            return function;
        }
    } // namespace

    CryptoModule::~CryptoModule()
    {
        if (handle == nullptr) return;
#ifdef _WIN32
        FreeLibrary(static_cast<HMODULE>(handle));
#else
        dlclose(handle);
#endif
    }

    void load_module(CryptoModule& module, const std::string& library_base_name)
    {
        module.handle = open_library(library_file_name(library_base_name));

        module.get_algorithm_info = reinterpret_cast<decltype(module.get_algorithm_info)>(
            resolve(module.handle, "get_algorithm_info"));
        module.get_output_size = reinterpret_cast<decltype(module.get_output_size)>(
            resolve(module.handle, "get_output_size"));
        module.generate_key
            = reinterpret_cast<decltype(module.generate_key)>(resolve(module.handle, "generate_key"));
        module.encrypt = reinterpret_cast<decltype(module.encrypt)>(resolve(module.handle, "encrypt"));
        module.decrypt = reinterpret_cast<decltype(module.decrypt)>(resolve(module.handle, "decrypt"));
    }
}

```

```

}
} // namespace cryptum

```

## Листинг В.15 — src/main.cpp

```

#include "algorithm_registry.h"
#include "byte_io.h"
#include "cli_options.h"
#include "crypto_module.h"
#include "pipeline.h"
#include "platform.h"

#include <exception>
#include <iostream>
#include <stdexcept>

namespace
{
void print_help()
{
    std::cout << "Usage: cryptum --algorithm NAME --mode MODE [options]\n"
                "\n"
                "Options:\n"
                "  -h, --help           print this help and exit\n"
                "  -a, --algorithm NAME algorithm to use\n"
                "  -m, --mode MODE      encrypt, decrypt or generate-key\n"
                "  -k, --key PATH       read the key from PATH, '-' for standard input\n"
                "  -i, --input PATH     read the data from PATH, '-' or omitted for "
                "standard input\n"
                "  -o, --output PATH    write the data to PATH, '-' or omitted for "
                "standard output\n"
                "  -g, --generate-key   generate a fresh key and use it for this operation\n"
                "  -s, --save-key PATH  write the key in use to PATH, '-' for standard "
                "output\n"
                "  -w, --write-key      equivalent to --save-key -\n"
                "\n"
                "An option takes its argument as '--option value', '--option=value' or\n"
                "'-o value'; combined short options and attached short values are not "
                "supported.\n"
                "\n"
                "Algorithms:\n"
                " "
                << cryptum::supported_algorithms() << "\n";
}

void run(const cryptum::CliOptions& options)
{
    cryptum::CryptoModule module;
    cryptum::load_module(module, cryptum::library_base_name(options.algorithm));

    const AlgorithmInfo* info = module.get_algorithm_info();
    if (info == nullptr || info->algorithm_name == nullptr || info->key_size == 0)
        throw std::runtime_error("the algorithm library reports no usable metadata");

    cryptum::SecureBuffer key(info->key_size);
    if (options.mode == cryptum::MODE_GENERATE_KEY || options.generate_key)
        cryptum::run_key_generation(module.generate_key, key);
    else
        cryptum::read_key(options.key_path, info->algorithm_name, key.bytes.data(),
                           key.bytes.size());

    if (options.has_save_key)
        cryptum::write_key(options.save_key_path, key.bytes.data(), key.bytes.size());
    if (options.mode == cryptum::MODE_GENERATE_KEY) return;

    std::ifstream input_file;
    std::ofstream output_file;
    std::istream& input = cryptum::open_input(options.input_path, input_file);
    std::ostream& output = cryptum::open_output(options.output_path, output_file);

    const OperationType operation
        = options.mode == cryptum::MODE_ENCRYPT ? OPERATION_ENCRYPT : OPERATION_DECRYPT;
    cryptum::run_transformation(operation == OPERATION_ENCRYPT ? module.encrypt : module.decrypt,
                                module.get_output_size, operation,
                                ConstBuffer{key.bytes.data(), key.bytes.size()}, input, output);
}

} // namespace

int main(int argument_count, char** arguments)
{
    try
    {
        cryptum::initialise_platform();
    }
}

```

```

        const cryptum::CliOptions options = cryptum::parse_command_line(argument_count, arguments);
        if (options.mode == cryptum::MODE_HELP) print_help();
        else run(options);
        return 0;
    }
    catch (const std::exception& error)
    {
        std::cerr << "cryptum: " << error.what() << "\n";
        return 1;
    }
}

```

## Листинг В.16 — src/pipeline.h

```

#ifndef CRYPTUM_PIPELINE_H
#define CRYPTUM_PIPELINE_H

#include "crypto_abi.h"
#include "secure_memory.h"

#include <cstdint>
#include <cstdlib>
#include <istream>
#include <ostream>

namespace cryptum
{
    using GenerateKeyFunction = int (*)(MutBuffer*);
    using OutputSizeFunction = std::size_t (*)(std::size_t, int);
    using TransformFunction = int (*)(ConstBuffer, ConstBuffer, MutBuffer*, std::uint64_t);

    void run_key_generation(GenerateKeyFunction generate, SecureBuffer& key);
    void run_transformation(TransformFunction transform, OutputSizeFunction output_size,
                           OperationType operation, ConstBuffer key, std::istream& input,
                           std::ostream& output);
} // namespace cryptum

#endif

```

## Листинг В.17 — src/pipeline.cpp

```

#include "pipeline.h"

#include "byte_io.h"

#include <stdexcept>
#include <string>

namespace cryptum
{
    namespace
    {
        constexpr std::size_t CHUNK_SIZE = 64 * 1024;

        void check_status(int status, const std::string& operation)
        {
            switch (status)
            {
            case CRYPTO_OK:
                return;
            case CRYPTO_ERROR_INVALID_KEY:
                throw std::runtime_error(operation + " failed: the key is not valid for this algorithm");
            case CRYPTO_ERROR_INVALID_BUFFER:
                throw std::runtime_error(operation
                                         + " failed: the buffer passed to the algorithm is too"
                                         + " small");
            case CRYPTO_ERROR_RANDOM_FAILURE:
                throw std::runtime_error(operation + " failed: the secure random source is unavailable");
            default:
                throw std::runtime_error(operation
                                         + " failed: the algorithm library reported an internal"
                                         + " error");
            }
        }
    } // namespace

    void run_key_generation(GenerateKeyFunction generate, SecureBuffer& key)
    {
        MutBuffer target(key.bytes.data(), key.bytes.size());
    }
}

```

```

    check_status(generate(&target), "key generation");
    if (target.size != key.bytes.size())
        throw std::runtime_error("key generation failed: the algorithm produced a key of"
                                   " unexpected length");
}

void run_transformation(TransformFunction transform, OutputSizeFunction output_size,
                        OperationType operation, ConstBuffer key, std::istream& input,
                        std::ostream& output)
{
    const std::string operation_name = operation == OPERATION_ENCRYPT ? "encryption" : "decryption";

    SecureBuffer input_chunk(CHUNK_SIZE);
    SecureBuffer output_chunk(output_size(CHUNK_SIZE, operation));
    std::uint64_t stream_offset = 0;

    for (;;)
    {
        const std::size_t taken = read_bytes(input, input_chunk.bytes.data(), CHUNK_SIZE);
        if (taken == 0) break;

        const ConstBuffer source{input_chunk.bytes.data(), taken};
        MutBuffer target{output_chunk.bytes.data(), output_chunk.bytes.size()};
        check_status(transform(key, source, &target, stream_offset), operation_name);
        write_bytes(output, target.data, target.size);

        stream_offset += taken;
    }

    output.flush();
    if (!output) throw std::runtime_error("writing the output failed");
}
} // namespace cryptum

```

## Листинг В.18 — src/platform.h

```

#ifndef CRYPTUM_PLATFORM_H
#define CRYPTUM_PLATFORM_H

namespace cryptum
{

void initialise_platform();

}

#endif

```

## Листинг В.19 — src/platform.cpp

```

#include "platform.h"

#ifdef _WIN32
#include <cstdio>
#include <fcntl.h>
#include <io.h>
#include <windows.h>
#endif

namespace cryptum
{

void initialise_platform()
{
#ifdef _WIN32
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);
    _setmode(_fileno(stdin), _O_BINARY);
    _setmode(_fileno(stdout), _O_BINARY);
#endif
}

} // namespace cryptum

```

## Листинг В.20 — lib/beaufort/beaufort.cpp

```

#include "crypto_abi.h"
#include "random_bytes.h"

#include <algorithm>
#include <cstdint>
#include <cstdlib>

```

```

#include <stdexcept>

namespace
{
    constexpr std::size_t KEY_SIZE = 16;

    const AlgorithmInfo ALGORITHM_INFO = {"beaufort", KEY_SIZE};

    bool is_valid_key(ConstBuffer key) { return key.data != nullptr && key.size == KEY_SIZE; }
} // namespace

extern "C"
{
    const AlgorithmInfo* get_algorithm_info(void) { return &ALGORITHM_INFO; }

    size_t get_output_size(size_t input_size, int) { return input_size; }

    // Every sequence of sixteen bytes is a valid key, so the random bytes are used as they come.
    int generate_key(MutBuffer* key)
    {
        try
        {
            if (key == nullptr || key->data == nullptr || key->size < KEY_SIZE)
                return CRYPTO_ERROR_INVALID_BUFFER;

            cryptum::fill_random_bytes(key->data, KEY_SIZE);
            key->size = KEY_SIZE;
            return CRYPTO_OK;
        }
        catch (const std::runtime_error&)
        {
            return CRYPTO_ERROR_RANDOM_FAILURE;
        }
        catch (...)
        {
            return CRYPTO_ERROR_INTERNAL;
        }
    }

    // C[i] = K[i mod 16] - P[i] (mod 256), where i is the absolute index of the byte in the
    // stream; it is taken from the address of the byte so that the lambda needs no state.
    int encrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t stream_offset)
    {
        try
        {
            if (output == nullptr) return CRYPTO_ERROR_INVALID_BUFFER;
            if (!is_valid_key(key)) return CRYPTO_ERROR_INVALID_KEY;
            if (output->size < input.size) return CRYPTO_ERROR_INVALID_BUFFER;

            std::transform(input.data, input.data + input.size, output->data,
                [key, input, stream_offset](const std::uint8_t& value) {
                    const std::uint64_t index
                        = stream_offset
                          + static_cast<std::uint64_t>(&value - input.data);
                    return static_cast<std::uint8_t>(key.data[index % KEY_SIZE] - value);
                });
            output->size = input.size;
            return CRYPTO_OK;
        }
        catch (...)
        {
            return CRYPTO_ERROR_INTERNAL;
        }
    }

    // The transformation is an involution, so decryption is the very same operation.
    int decrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t stream_offset)
    {
        return encrypt(key, input, output, stream_offset);
    }
}

```

## Листинг B.21 — lib/substitution/substitution.cpp

```

#include "crypto_abi.h"
#include "random_bytes.h"
#include "secure_memory.h"

#include <algorithm>
#include <array>
#include <cstdint>
#include <cstdint>
#include <limits>

```

```

#include <numeric>
#include <random>
#include <stdexcept>

namespace
{
    constexpr std::size_t KEY_SIZE = 256;

    const AlgorithmInfo ALGORITHM_INFO = {"simple substitution", KEY_SIZE};

    bool is_valid_key(ConstBuffer key)
    {
        if (key.data == nullptr || key.size != KEY_SIZE) return false;

        std::array<std::uint8_t, KEY_SIZE> occurrences{};
        std::for_each(key.data, key.data + key.size,
            [&occurrences](std::uint8_t value) { ++occurrences[value]; });
        return std::all_of(occurrences.begin(), occurrences.end(),
            [](std::uint8_t count) { return count == 1; });
    }

    // A uniform random bit generator is what std::shuffle expects; this one draws from the
    // system random source instead of a seeded pseudo random engine.
    struct SecureRandomEngine
    {
        using result_type = std::uint32_t;

        static constexpr result_type min() { return 0; }
        static constexpr result_type max() { return std::numeric_limits<result_type>::max(); }

        result_type operator()() const
        {
            result_type value = 0;
            cryptum::fill_random_bytes(reinterpret_cast<std::uint8_t*>(&value), sizeof(value));
            return value;
        }
    };

    void apply_table(const std::uint8_t* table, ConstBuffer input, MutBuffer* output)
    {
        std::transform(input.data, input.data + input.size, output->data,
            [table](std::uint8_t value) { return table[value]; });
        output->size = input.size;
    }
} // namespace

extern "C"
{
    const AlgorithmInfo* get_algorithm_info(void) { return &ALGORITHM_INFO; }

    size_t get_output_size(size_t input_size, int) { return input_size; }

    int generate_key(MutBuffer* key)
    {
        try
        {
            if (key == nullptr || key->data == nullptr || key->size < KEY_SIZE)
                return CRYPTO_ERROR_INVALID_BUFFER;

            std::iota(key->data, key->data + KEY_SIZE, std::uint8_t{0});
            std::shuffle(key->data, key->data + KEY_SIZE, SecureRandomEngine{});
            key->size = KEY_SIZE;
            return CRYPTO_OK;
        }
        catch (const std::runtime_error&)
        {
            return CRYPTO_ERROR_RANDOM_FAILURE;
        }
        catch (...)
        {
            return CRYPTO_ERROR_INTERNAL;
        }
    }

    // The substitution is position independent, so the stream offset is not used here.
    int encrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t)
    {
        try
        {
            if (output == nullptr) return CRYPTO_ERROR_INVALID_BUFFER;
            if (!is_valid_key(key)) return CRYPTO_ERROR_INVALID_KEY;
            if (output->size < input.size) return CRYPTO_ERROR_INVALID_BUFFER;

            apply_table(key.data, input, output);
            return CRYPTO_OK;
        }
    }
}

```

```

    }
    catch (...)
    {
        return CRYPTO_ERROR_INTERNAL;
    }
}

int decrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t)
{
    try
    {
        if (output == nullptr) return CRYPTO_ERROR_INVALID_BUFFER;
        if (!is_valid_key(key)) return CRYPTO_ERROR_INVALID_KEY;
        if (output->size < input.size) return CRYPTO_ERROR_INVALID_BUFFER;

        cryptum::SecureBuffer inverse(KEY_SIZE);
        for (std::size_t value = 0; value < KEY_SIZE; ++value)
            inverse.bytes[key.data[value]] = static_cast<std::uint8_t>(value);

        apply_table(inverse.bytes.data(), input, output);
        return CRYPTO_OK;
    }
    catch (...)
    {
        return CRYPTO_ERROR_INTERNAL;
    }
}
}

```

## Листинг В.22 — lib/trithemius/trithemius.cpp

```

#include "crypto_abi.h"
#include "random_bytes.h"

#include <algorithm>
#include <cstdint>
#include <cstddef>
#include <stdexcept>

namespace
{
    constexpr std::size_t KEY_SIZE = 16;

    const AlgorithmInfo ALGORITHM_INFO = {"trithemius", KEY_SIZE};

    bool is_valid_key(ConstBuffer key) { return key.data != nullptr && key.size == KEY_SIZE; }

    std::uint64_t absolute_index(const std::uint8_t& value, ConstBuffer input, uint64_t stream_offset)
    {
        return stream_offset + static_cast<std::uint64_t>(&value - input.data);
    }
} // namespace

extern "C"
{
    const AlgorithmInfo* get_algorithm_info(void) { return &ALGORITHM_INFO; }

    size_t get_output_size(size_t input_size, int) { return input_size; }

    // Every sequence of sixteen bytes is a valid key, so the random bytes are used as they come.
    int generate_key(MutBuffer* key)
    {
        try
        {
            if (key == nullptr || key->data == nullptr || key->size < KEY_SIZE)
                return CRYPTO_ERROR_INVALID_BUFFER;

            cryptum::fill_random_bytes(key->data, KEY_SIZE);
            key->size = KEY_SIZE;
            return CRYPTO_OK;
        }
        catch (const std::runtime_error&)
        {
            return CRYPTO_ERROR_RANDOM_FAILURE;
        }
        catch (...)
        {
            return CRYPTO_ERROR_INTERNAL;
        }
    }

    // C[i] = P[i] + K[i mod 16] + i (mod 256), where i is the absolute index of the byte in the
    // stream; it is taken from the address of the byte so that the lambda needs no state.

```

```

int encrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t stream_offset)
{
    try
    {
        if (output == nullptr) return CRYPTO_ERROR_INVALID_BUFFER;
        if (!is_valid_key(key)) return CRYPTO_ERROR_INVALID_KEY;
        if (output->size < input.size) return CRYPTO_ERROR_INVALID_BUFFER;

        std::transform(
            input.data, input.data + input.size, output->data,
            [key, input, stream_offset](const std::uint8_t& value) {
                const std::uint64_t index = absolute_index(value, input, stream_offset);
                return static_cast<std::uint8_t>(value + key.data[index % KEY_SIZE] + index);
            });
        output->size = input.size;
        return CRYPTO_OK;
    }
    catch (...)
    {
        return CRYPTO_ERROR_INTERNAL;
    }
}

int decrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t stream_offset)
{
    try
    {
        if (output == nullptr) return CRYPTO_ERROR_INVALID_BUFFER;
        if (!is_valid_key(key)) return CRYPTO_ERROR_INVALID_KEY;
        if (output->size < input.size) return CRYPTO_ERROR_INVALID_BUFFER;

        std::transform(
            input.data, input.data + input.size, output->data,
            [key, input, stream_offset](const std::uint8_t& value) {
                const std::uint64_t index = absolute_index(value, input, stream_offset);
                return static_cast<std::uint8_t>(value - key.data[index % KEY_SIZE] - index);
            });
        output->size = input.size;
        return CRYPTO_OK;
    }
    catch (...)
    {
        return CRYPTO_ERROR_INTERNAL;
    }
}
}

```

## Листинг В.23 — tests/test\_algorithms.cpp

```

#include "crypto_module.h"

#include <algorithm>
#include <cstdint>
#include <cstdint>
#include <iostream>
#include <iterator>
#include <random>
#include <stdexcept>
#include <string>
#include <vector>

namespace
{
    using Bytes = std::vector<std::uint8_t>;
    using TransformFunction = int (*)(ConstBuffer, ConstBuffer, MutBuffer*, std::uint64_t);

    constexpr std::uint32_t SEED = 20260915;

    struct TestCase
    {
        const char* name;
        void (*run)();
    };

    void require(bool condition, const std::string& message)
    {
        if (!condition) throw std::runtime_error(message);
    }

    Bytes pseudo_random_bytes(std::size_t size)
    {
        std::mt19937 engine(SEED);
        Bytes data(size);
    }
}

```

```

    std::generate(data.begin(), data.end(),
                  [&engine] { return static_cast<std::uint8_t>(engine()); });
    return data;
}

Bytes apply(TransformFunction transform, const Bytes& key, const Bytes& input,
            std::uint64_t stream_offset)
{
    Bytes output(input.size());
    MutBuffer target{output.data(), output.size()};
    const int status = transform(ConstBuffer{key.data(), key.size()},
                                ConstBuffer{input.data(), input.size()}, &target, stream_offset);
    require(status == CRYPTO_OK, "the algorithm returned status " + std::to_string(status));
    output.resize(target.size());
    return output;
}

Bytes generated_key(const cryptum::CryptoModule& module)
{
    const AlgorithmInfo* info = module.get_algorithm_info();
    Bytes key(info->key_size);
    MutBuffer target(key.data(), key.size());
    require(module.generate_key(&target) == CRYPTO_OK, "the key could not be generated");
    require(target.size() == info->key_size, "the generated key has the wrong length");
    return key;
}

void check_round_trip(const char* library, const Bytes& plaintext)
{
    cryptum::CryptoModule module;
    cryptum::load_module(module, library);

    const Bytes key = generated_key(module);
    const Bytes ciphertext = apply(module.encrypt, key, plaintext, 0);
    require(ciphertext.size() == plaintext.size(), "the ciphertext has a different length");
    require(apply(module.decrypt, key, ciphertext, 0) == plaintext,
            "the decrypted data differs from the original");
}

void check_known_answer(const char* library, const Bytes& key, const Bytes& plaintext,
                        const Bytes& ciphertext)
{
    cryptum::CryptoModule module;
    cryptum::load_module(module, library);

    require(apply(module.encrypt, key, plaintext, 0) == ciphertext,
            "the ciphertext differs from the expected bytes");
    require(apply(module.decrypt, key, ciphertext, 0) == plaintext,
            "the recovered plaintext differs from the expected bytes");
}

Bytes reversing_key()
{
    Bytes key(256);
    for (std::size_t value = 0; value < key.size(); ++value)
        key[value] = static_cast<std::uint8_t>(255 - value);
    return key;
}

Bytes sixteen_byte_key()
{
    return {0xA0, 0xB1, 0xC2, 0xD3, 0xE4, 0xF5, 0x06, 0x17,
            0x28, 0x39, 0x4A, 0x5B, 0x6C, 0x7D, 0x8E, 0x9F};
}

// The expected bytes come from the formulas themselves: the substitution key maps b to
// 255 - b, beaufort computes K[i mod 16] - P[i] and trithemius P[i] + K[i mod 16] + i.
void substitution_known_answer()
{
    check_known_answer("substitution", reversing_key(), {0x00, 0x01, 0x41, 0xFF},
                      {0xFF, 0xFE, 0xBE, 0x00});
}

void beaufort_known_answer()
{
    check_known_answer("beaufort", sixteen_byte_key(), {0x10, 0x20, 0x30, 0x41, 0x05},
                      {0x90, 0x91, 0x92, 0x92, 0xDF});
}

void trithemius_known_answer()
{
    check_known_answer("trithemius", sixteen_byte_key(), {0x10, 0x20, 0x30, 0x41, 0x05},
                      {0xB0, 0xD2, 0xF4, 0x17, 0xED});
}

void substitution_empty_input() { check_round_trip("substitution", Bytes()); }

```

```

void beaufort_empty_input() { check_round_trip("beaufort", Bytes()); }
void trithemius_empty_input() { check_round_trip("trithemius", Bytes()); }

void substitution_unaligned_input() { check_round_trip("substitution", pseudo_random_bytes(37)); }
void beaufort_unaligned_input() { check_round_trip("beaufort", pseudo_random_bytes(37)); }
void trithemius_unaligned_input() { check_round_trip("trithemius", pseudo_random_bytes(37)); }

void substitution_one_mebibyte()
{
    check_round_trip("substitution", pseudo_random_bytes(1024 * 1024));
}
void beaufort_one_mebibyte() { check_round_trip("beaufort", pseudo_random_bytes(1024 * 1024)); }
void trithemius_one_mebibyte() { check_round_trip("trithemius", pseudo_random_bytes(1024 * 1024)); }

void check_chunked_matches_single_call(const char* library)
{
    cryptum::CryptoModule module;
    cryptum::load_module(module, library);

    const Bytes key = generated_key(module);
    const Bytes plaintext = pseudo_random_bytes(1000);
    const Bytes whole = apply(module.encrypt, key, plaintext, 0);

    const std::size_t CHUNKS[] = {1, 7, 13, 64, 300, 615};
    Bytes chunked;
    std::size_t offset = 0;
    for (std::size_t size : CHUNKS)
    {
        const Bytes part(plaintext.begin() + static_cast<std::ptrdiff_t>(offset),
                        plaintext.begin() + static_cast<std::ptrdiff_t>(offset + size));
        const Bytes encrypted = apply(module.encrypt, key, part, offset);
        chunked.insert(chunked.end(), encrypted.begin(), encrypted.end());
        offset += size;
    }

    require(offset == plaintext.size(), "the chunks do not cover the whole input");
    require(chunked == whole, "chunk by chunk encryption differs from a single call");
}

void check_key_generation(const char* library)
{
    cryptum::CryptoModule module;
    cryptum::load_module(module, library);

    const AlgorithmInfo* info = module.get_algorithm_info();
    require(generated_key(module).size() == info->key_size,
           "the generated key does not have the declared length");

    Bytes too_small(info->key_size - 1);
    MutBuffer target(too_small.data(), too_small.size());
    require(module.generate_key(&target) == CRYPTO_ERROR_INVALID_BUFFER,
           "a buffer smaller than the key length was accepted");
}

void substitution_chunks() { check_chunked_matches_single_call("substitution"); }
void beaufort_chunks() { check_chunked_matches_single_call("beaufort"); }
void trithemius_chunks() { check_chunked_matches_single_call("trithemius"); }

void substitution_key_generation() { check_key_generation("substitution"); }
void beaufort_key_generation() { check_key_generation("beaufort"); }
void trithemius_key_generation() { check_key_generation("trithemius"); }

void substitution_rejects_invalid_keys()
{
    cryptum::CryptoModule module;
    cryptum::load_module(module, "substitution");

    const Bytes plaintext = {0x41, 0x42, 0x43};
    Bytes output(plaintext.size());
    MutBuffer target(output.data(), output.size());
    const ConstBuffer source(plaintext.data(), plaintext.size());

    const Bytes repeated(256, 0x00);
    require(module.encrypt(ConstBuffer{repeated.data(), repeated.size()}, source, &target, 0)
           == CRYPTO_ERROR_INVALID_KEY,
           "a key that is not a permutation was accepted");

    const Bytes wrong_length(16, 0x00);
    require(
        module.encrypt(ConstBuffer{wrong_length.data(), wrong_length.size()}, source, &target, 0)
        == CRYPTO_ERROR_INVALID_KEY,
        "a key of the wrong length was accepted");
}

const TestCase TESTS[] = {
    {"substitution: known answer", substitution_known_answer},

```

```

    {"substitution: empty input", substitution_empty_input},
    {"substitution: length that is not a multiple of the key length", substitution_unaligned_input},
    {"substitution: one mebibyte of pseudo random data", substitution_one_mebibyte},
    {"substitution: chunk by chunk equals a single call", substitution_chunks},
    {"substitution: generated key length", substitution_key_generation},
    {"beaufort: known answer", beaufort_known_answer},
    {"beaufort: empty input", beaufort_empty_input},
    {"beaufort: length that is not a multiple of the key length", beaufort_unaligned_input},
    {"beaufort: one mebibyte of pseudo random data", beaufort_one_mebibyte},
    {"beaufort: chunk by chunk equals a single call", beaufort_chunks},
    {"beaufort: generated key length", beaufort_key_generation},
    {"trithemius: known answer", trithemius_known_answer},
    {"trithemius: empty input", trithemius_empty_input},
    {"trithemius: length that is not a multiple of the key length", trithemius_unaligned_input},
    {"trithemius: one mebibyte of pseudo random data", trithemius_one_mebibyte},
    {"trithemius: chunk by chunk equals a single call", trithemius_chunks},
    {"trithemius: generated key length", trithemius_key_generation},
    {"substitution: a key that is not a permutation is rejected",
     substitution_rejects_invalid_keys},
};
} // namespace

int main()
{
    const int total    = static_cast<int>(std::size(TESTS));
    int          failures = 0;

    for (const TestCase& test : TESTS)
    {
        try
        {
            test.run();
            std::cout << "pass  " << test.name << "\n";
        }
        catch (const std::exception& error)
        {
            std::cout << "FAIL  " << test.name << ": " << error.what() << "\n";
            ++failures;
        }
    }

    std::cout << total - failures << " of " << total << " tests passed\n";
    return failures == 0 ? 0 : 1;
}

```

## Листинг B.24 — scripts/install.sh

```

#!/usr/bin/env bash
set -euo pipefail

LIBRARY_DIRECTORY=/usr/local/lib/cryptum
EXECUTABLE_LINK=/usr/local/bin/cryptum
LIBRARIES=(libsubstitution.so libbeaufort.so libtrithemius.so)

script_directory=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd)
source_directory=${1:-$script_directory/./build/bin}

if [ "$(id -u)" -ne 0 ]; then
    echo "install.sh: administrative rights are required" >&2
    exit 1
fi

for file in cryptum "${LIBRARIES[@]}; do
    if [ ! -f "$source_directory/$file" ]; then
        echo "install.sh: '$source_directory/$file' not found, build the project first" >&2
        exit 1
    fi
done

install -d -m 0755 "$LIBRARY_DIRECTORY"
install -m 0755 "$source_directory/cryptum" "$LIBRARY_DIRECTORY/cryptum"
for file in "${LIBRARIES[@]}; do
    install -m 0644 "$source_directory/$file" "$LIBRARY_DIRECTORY/$file"
done
ln -sfn "$LIBRARY_DIRECTORY/cryptum" "$EXECUTABLE_LINK"

echo "cryptum installed in $LIBRARY_DIRECTORY and linked as $EXECUTABLE_LINK"

```

## Листинг B.25 — scripts/uninstall.sh

```

#!/usr/bin/env bash
set -euo pipefail

```

```

LIBRARY_DIRECTORY=/usr/local/lib/cryptum
EXECUTABLE_LINK=/usr/local/bin/cryptum
LIBRARIES=(libsubstitution.so libbeaufort.so libtrithemius.so)

if [ "$(id -u)" -ne 0 ]; then
    echo "uninstall.sh: administrative rights are required" >&2
    exit 1
fi

if [ -L "$EXECUTABLE_LINK" ] && [ "$(readlink "$EXECUTABLE_LINK")" = "$LIBRARY_DIRECTORY/cryptum" ]; then
    rm -f "$EXECUTABLE_LINK"
fi

for file in cryptum "${LIBRARIES[@]}; do
    rm -f "$LIBRARY_DIRECTORY/$file"
done
rmdir "$LIBRARY_DIRECTORY" 2>/dev/null || true

echo "cryptum removed"

```

## Листинг В.26 — scripts/install.ps1

```

param([string]$SourceDirectory = (Join-Path $PSScriptRoot '..\build\bin'))

$ErrorActionPreference = 'Stop'

$InstallDirectory = Join-Path $env:LOCALAPPDATA 'cryptum'
$Files = @('cryptum.exe', 'substitution.dll', 'beaufort.dll', 'trithemius.dll')

foreach ($file in $Files) {
    if (-not (Test-Path (Join-Path $SourceDirectory $file))) {
        throw "'$file' not found in '$SourceDirectory', build the project first"
    }
}

New-Item -ItemType Directory -Force -Path $InstallDirectory | Out-Null
foreach ($file in $Files) {
    Copy-Item -Force -Path (Join-Path $SourceDirectory $file) -Destination $InstallDirectory
}

$userPath = [Environment]::GetEnvironmentVariable('Path', 'User')
if ([string]::IsNullOrEmpty($userPath)) {
    [Environment]::SetEnvironmentVariable('Path', $InstallDirectory, 'User')
}
elseif ($userPath.Split(';') -notcontains $InstallDirectory) {
    [Environment]::SetEnvironmentVariable('Path', "$userPath;$InstallDirectory", 'User')
}

Write-Output "cryptum installed in $InstallDirectory"

```

## Листинг В.27 — scripts/uninstall.ps1

```

$ErrorActionPreference = 'Stop'

$InstallDirectory = Join-Path $env:LOCALAPPDATA 'cryptum'

if (Test-Path $InstallDirectory) {
    Remove-Item -Recurse -Force -Path $InstallDirectory
}

$userPath = [Environment]::GetEnvironmentVariable('Path', 'User')
if (-not [string]::IsNullOrEmpty($userPath)) {
    $entries = $userPath.Split(';') | Where-Object { $_ -ne $InstallDirectory -and $_ -ne '' }
    [Environment]::SetEnvironmentVariable('Path', ($entries -join ';'), 'User')
}

Write-Output 'cryptum removed'

```